SWE5006 Practice Project · Graduate Certificate in Designing Modern Software Systems

Speaker Script and Live Demo Run Book

BlocBuy — Team 8 · Monday 26 October 2026, 16:30–17:10 SGT (Zoom)

Responsible	Yu Qianshuo (Yorik), presentation lead and storyline (#464); Lim Jiechao, slide master.
Specification	`docs/swe5006/presentation/RUBRIC.md`: every slide records the rubric rows it covers; the coverage tables in the run sheet are derived from them.
Companion files	`swe5006-presentation.html` — the deck (39 slides + backups BX, B1–B16, the 1:1 diagram detail pages `D9-1` … `D31-3` (index DX) and the full-size diagrams F1–F21 (index FX); Chrome or Edge; **S** notes, **T** timer, **O** overview, **F** fullscreen, **B** backup index) · `swe5006-presentation.pdf` — the same deck for submission.
Facts as of	25 Oct 2026. Dev health `UP` (23 Oct); a paid START and a natural expiry refund shown on dev on 23 Oct (PR #664, merged 24 Oct, bef39b72); a second buyer's paid JOIN shown on dev on 24 Oct (system demo backup recording, #657); #590, #517 and #223 closed by Yorik on 25 Oct; #224 open.

Markers

`[FILL]` value that does not exist yet; never present a guess · `[VERIFY]` claim to check against the repository, report or run logs · `[CONFIRM]` needs team agreement. All must be gone by the 25 Oct rehearsal.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:30	5 s	1	BlocBuy	Lim	1a	4 s
16:30:05	25 s	2	Group buying fails when buyers cannot commit together	Lim	1a	24 s
16:30:30	25 s	3	Scope: nine use cases, six quality attributes with targets	Lim	1b	23 s
16:30:55	35 s	4	Promised, delivered, changed, dropped: status on 24 October	Lim	1c	34 s
16:31:30	20 s	5	Milestones achieved and effort per member	Lim	1d	18 s
16:31:50	10 s	6	Ownership: at least one use case and one design problem each	Lim	1d	6 s
16:32	35 s	7	Four sprints: the Definition of Done reached, and what each retro changed	Lim	1e	33 s
16:32:35	25 s	8	Agile artefacts: a story card, a burndown, a practice	Lim	1g	23 s
16:33	32 s	9	Logical architecture: COLA layers across six bounded contexts	Eric	2a	28 s
16:33:32	30 s	10	Transition strategies from analysis to design	Eric	2b	27 s
16:34:02	24 s	11	Typical changes: class and sequence diagrams	Eric	2b	21 s
16:34:26	20 s	12	One strategy pair: analysis objects become design classes	Eric	2b	18 s
16:34:46	30 s	13	Use case diagram: in scope, out of scope and status	Eric	2c	28 s
16:35:16	40 s	14	UC4 join: the analysis model	Eric	2d	36 s
16:35:56	29 s	15	UC4 join: the design class diagram	Eric	2d	25 s
16:36:25	47 s	16	UC4 join: the design sequence, paid join by webhook	Eric	2d	43 s
16:37:12	10 s	17	Team-formation statechart	Eric	2d	9 s
16:37:22	48 s	18	UC3 start a team: Factory Method	Eric	2e	41 s
16:38:10	30 s	19	UC5 to UC6 team outcome: Observer	Yorik	2e	30 s
16:38:40	20 s	20	Notifications: Template Method for outbox transports	Pan	2e	18 s
16:39	35 s	21	UC1 pricing: Strategy	Lim	2e	34 s
16:39:35	22 s	22	Design problem: listing rules that depend on its state	Liu Jin	2e	20 s
16:39:57	19 s	23	After: the State pattern	Liu Jin	2e	17 s
16:40:16	15 s	24	Database schema: core entities and mapping decisions	Liu Jin	2f	13 s
16:38	38 s	25	Testing on the Agile Testing Quadrants	Liu Jin	3a	34 s
16:38:38	30 s	26	Scripted performance test: tool, scenario, target	Liu Jin	3a	30 s
16:41	38 s	27	Source code: one monorepo, Branch by Feature, reviewed changes	Pan	4a	37 s
16:41:38	30 s	28	Artefacts: each one traceable to a commit	Pan	4a	29 s
16:42:08	46 s	29	Continuous integration: pull-request jobs and post-merge jobs	Pan	4b	40 s
16:42:54	48 s	30	Security controls by pipeline stage, and what each one catches	Pan	4d	48 s

16:43:42	57 s	31	Delivery: one environment, the code owner approves, the deploy verifies itself	Pan	4b	54 s
16:44:39	51 s	32	Technical findings: three issues, their root causes and fixes	Pan	4c	51 s
16:45:30	330 s	33	Live: one change from pull request to dev	Pan	4e	17 s
16:51	30 s	34	System demo: two buyers, one bloc, and the qualities it shows	Yorik	5a	17 s
16:51:30	510 s	35	On dev: two buyers, one bloc	Yorik	5c	31 s
17:00	120 s	36	Performance against the target: the scripted k6 run	Yorik	5b	19 s
17:02	30 s	37	Scalability: a stated trade-off, not a claim	Yorik	5b	27 s
17:02:30	30 s	38	Maintainability: the pipeline you just saw, and the seams in the code	Yorik	5b	27 s
17:03	Q&A	39	Thank you	Yorik	5b	

Segment talk budget vs slots vs estimated speaking time: §1 180 s / 180 s / 165 s · §2 300 s / 451 s / 408 s · §3 180 s / 68 s / 64 s · §4 600 s / 600 s / 276 s · §5 720 s / 720 s / 121 s.

34 minutes scripted, 16:30–17:04; the 6 minutes to 17:10 are the buffer for questions asked during the talk (rubric G1–G3). §1 runs 3 minutes and §2 6 minutes so that each member presents their own design problem (rubric G4, time budget).

Interruption policy

- A lecturer may ask at any moment. The current speaker answers in **one or two sentences** (decision → reason → evidence or limitation), says “back to …” and continues from the same slide.
- If the owner is someone else, the speaker says so and Lim posts `Q: <question> → owner` in the backchannel; parked questions are answered first in the buffer.
- Hard checkpoints: 16:33 Eric, 16:39 Liu Jin, 16:42 Pan, 16:52 Yorik.

Handovers (rubric G4: rehearse each one)

From slide	Speakers
9	Lim Jiechao → Wang Ziang (Eric)
19	Wang Ziang (Eric) → Yu Qianshuo (Yorik)
20	Yu Qianshuo (Yorik) → Panpakorn Siripanich (Pan)
21	Panpakorn Siripanich (Pan) → Lim Jiechao
22	Lim Jiechao → Liu Jin
27	Liu Jin → Panpakorn Siripanich (Pan)
34	Panpakorn Siripanich (Pan) → Yu Qianshuo (Yorik)

Every guideline item has slides and a speaker

Item	Guideline requirement (verbatim)	Slides	Speaker
1a	Explain the context and the business problem that the solution as a whole is trying to achieve.	1, 2	Lim Jiechao
1b	Explain the scope of the project.	3	Lim Jiechao

1c	Explain the current status of the project including outstanding issues.	4	Lim Jiechao
1d	Describe the actual project milestones and the total effort expended by each team member.	5, 6	Lim Jiechao
1e	Explain how many sprints planned, Definition of Done (DoD) achieved.	7	Lim Jiechao
1f	Sprint retrospective & reflection in the next sprint	7	Lim Jiechao
1g	Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.	8	Lim Jiechao
2a	Software architectural decisions that influence the logical architecture.	9	Wang Ziang
2b	Transition from analysis to design	10, 11, 12	Wang Ziang
2c	Use case diagram	13	Wang Ziang
2d	Analysis and design models	14, 15, 16, 17	Wang Ziang
2e	Design problems and patterns	18, 19, 20, 21, 22, 23	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2f	Database schemas	24	Liu Jin
3a	Types of Testing Conducted	25, 26	Liu Jin
3b	Well structured source code with unit test cases for classes with more complex behaviours.	25	Liu Jin
4a	Describe the strategy to manage the source code and artifacts.	27, 28	Panpakorn Siripanich
4b	Describe the DevSecOps pipeline and procedures.	29, 31	Panpakorn Siripanich
4c	Highlight any technical findings and issues encountered.	32	Panpakorn Siripanich
4d	Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.	30	Panpakorn Siripanich
4e	Backup: Record a video in advance in case of any technical issues during the live demo.	33	Panpakorn Siripanich
5a	Demonstrate some of the significant use cases of the solution.	34	Yu Qianshuo
5b	Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).	36, 37, 38, 39	Yu Qianshuo
5c	Backup: Record a video in advance in case of any technical issues during the live demo.	35	Yu Qianshuo

Every rubric row has a slide (rubric checklist)

Rubric row	Slides	Speaker
1.1	1, 2	Lim Jiechao
1.2	3	Lim Jiechao
1.3	4	Lim Jiechao
1.4	5	Lim Jiechao
1.5	6	Lim Jiechao

1.6	7	Lim Jiechao
1.7	7	Lim Jiechao
1.8	8	Lim Jiechao
2.1	9	Wang Ziang
2.2	10, 11, 12, 15, 16	Wang Ziang
2.3	13	Wang Ziang
2.4	12, 14, 15, 16, 17, 23	Wang Ziang, Liu Jin
2.5	18, 19, 20, 21, 22, 23	Wang Ziang, Yu Qianshuo, Panpakorn Siripanich, Lim Jiechao, Liu Jin
2.6	24	Liu Jin
3.1	25	Liu Jin
3.2	26	Liu Jin
3.3	25, 30	Liu Jin, Panpakorn Siripanich
4.1	33	Panpakorn Siripanich
4.2	27, 28	Panpakorn Siripanich
4.3	29, 30	Panpakorn Siripanich
4.4	31	Panpakorn Siripanich
4.5	32	Panpakorn Siripanich
4.6	33	Panpakorn Siripanich
5.1	34, 35	Yu Qianshuo
5.2	34, 35, 37	Yu Qianshuo
5.3	36	Yu Qianshuo
5.4	38	Yu Qianshuo
5.5	35	Yu Qianshuo

Presentation tips checklist

Tip (guidelines page 2)	How this deck meets it
Summarize and highlight the significant achievements. Talk more about the significant and interesting parts of the solution and implementation. Your audience should remember the unique aspects of the solution and not similarities to other solutions.	Depth goes to what is unique: pay-first teams that refund themselves, money safety under concurrency and failure (UC4 design, statechart, design problems), and a pipeline from pull request to dev in minutes.
Manage your time well. Do not present too many slides. Remember that the audience can read the text in the slide so refrain from reading them.	39 main slides in 34 minutes, two of them live demos; slots checked at 130 wpm in the table above; sample wording adds to the slide instead of reading it; details sit in backups B1–B16.
Explain your design in top-down manner. Make sure that your audience understands the requirements, the high-level design and the high-level decisions that you make before drilling into some details.	Problem, scope and quality attributes first (slides 2–3); then architecture (9), transition strategies (10–12), use cases (13), one use case from analysis to design with its statechart (14–17), design problems (18–23), schema (24).
Ensure that the contents on the slides are readable from the distance at which your audience is sitting. This is particular true for face-to-face presentation.	Rubric V1–V7: one idea per slide, diagrams cropped to the part discussed, labels checked on a projector or TV before 26 Oct.

Cue words in the team backchannel (never the Zoom chat)

Message	Meaning	Sent by
`1 MIN`	One minute of talk left in this segment	Lim
`TIME`	Stop; finish the sentence and hand over	Lim
`Q: <text> → name`	A question was parked; the named owner answers it in the buffer (17:04–17:10)	Lim
`BEHIND n / AHEAD n`	The talk is n seconds behind or ahead of the run sheet; the next speaker applies the trims	Lim
`SHARE`	Next person: start screen sharing now	Lim
`CLIP n`	Live step n failed; play the recording of step n	Pan or Yorik
`EXPIRY OK / REFUND OK / REFUND LATE`	State of the 16:05 expiry bloc (16:10 and 16:50)	Yorik

1 Project Introduction & Project Management Summary (3+0 minutes)

Clock	16:30:00 – 16:33:00 (3 min talk + 0 min Q&A; talk ends 16:33:00)
Owner	Lim Jiechao
Slides	1, 2, 3, 4, 5, 6, 7, 8

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)
1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Explain the scope of the project.
1c — Explain the current status of the project including outstanding issues.
1d — Describe the actual project milestones and the total effort expended by each team member.
1e — Explain how many sprints planned, Definition of Done (DoD) achieved.
1f — Sprint retrospective & reflection in the next sprint
1g — Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.

Tip
Lim opens at 16:30:00 sharp, with the deck shared and the notes window off the shared screen. Eight slides in three minutes: the tables carry the facts, the wording carries only the point of each slide.

Slide 1 BlocBuy

16:30:00–16:30:05 · 5 s

Speaker Lim · covers guideline item **1a**

Say *Good afternoon. We're Team 8, and this is BlocBuy.*

≈ 9 words · ≈ 4 s at 130 wpm · slot 5 s

Cues

- 16:30:00 — Lim opens swe5006-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen) and presses T in either window to start the talk timer. In Zoom, share only the deck window.
- Backchannel: the team chat stays open. Lim repeats any question asked during the talk in one line and names who answers; the buffer is 17:04–17:10 (rubric G3).
- The roadmap line on the slide is the five sections of the lecturer's sequence; do not read it.

Hand-over *"The problem first."*

Slide 2 Group buying fails when buyers cannot commit together

16:30:05–16:30:30 · 25 s

Speaker Lim · covers guideline item **1a**

Say *This is what we're building, in Silicon Valley speak: early Pinduoduo, but for Singapore. And the problem it solves: the discount needs a crowd, and nobody wants to go first. By pairing upfront payment with automatic refund, we make going first safe. And an invite link makes sharing with friends easy.*

≈ 51 words · ≈ 24 s at 130 wpm · slot 25 s · if short of time: If over time, drop "And an invite link makes sharing with friends easy."

Cues

- Source: proposal §4 (27 Aug): the pay-first mechanic, two prices, automatic refund on expiry, modelled on the early Pinduoduo playbook; §6.1: Stripe test mode only.
- The one-sentence problem is the slide's headline: say it, then the mechanism. What makes it technical is that money is committed before the team exists, so every later section is about money safety under concurrency and failure.

Hand-over *"Here is what we set out to build."*

Slide 3 Scope: nine use cases, six quality attributes with targets

16:30:30–16:30:55 · 25 s

Speaker Lim · covers guideline item **1b**

Say *Within scope are the nine use cases as we proposed, plus in-app notifications for SWE5001. Out of scope stays as proposed. On the right are the six quality attributes we committed to, with their targets. Each is tagged with where we show it; the system demo proves the first three.*

≈ 50 words · ≈ 23 s at 130 wpm · slot 25 s · if short of time: If over time, drop "Out of scope stays as proposed."

Cues

- In scope: proposal §6.1, UC1–UC9. Receive notifications (#215, #446) was added in Sprint 4 for SWE5001; it is off on the EC2 dev host by decision (backend ADR 0007, 11 Oct).
- Out of scope: proposal §6.2 verbatim list (merchant self-service GUI; analytics; recommendation; automatic team matching; notifications beyond in-app; native apps; live payments). We went beyond the first item for admins only: an admin console (frontend ADR 0015), shown on slide 4.
- Quality attributes and targets: proposal §6.3, quoted. "Shown in" maps each to the rubric rows that demonstrate it: 5.2 (correctness, security), 5.3 (performance, k6 on the SWE5001 ECS environment), 5.4 and §2 (maintainability), §3–§4 (code quality, auditability).

Hand-over *"How much of that did we deliver?"*

Slide 4 Promised, delivered, changed, dropped: status on 24 October

16:30:55–16:31:30 · 35 s

Speaker Lim · covers guideline item **1c**

Say *All nine use cases are built, plus an admin console the proposal left out. On dev, paid start, a second buyer's join and the expiry refund have all been shown, by 24 October.* **[click 1/1]** *Eight things changed, and six still lack an ADR. The biggest: JWT became a cookie session, and dev runs on one EC2 host, not ECS. Production release lives only in our SWE5001 deployment. The security gaps are on the left.*

≈ 73 words · ≈ 34 s at 130 wpm · slot 35 s · if short of time: If over time, drop "plus an admin console the proposal left out".

Cues

- Use cases: #24–#32 all closed completed 19–28 Sep; UC8 shipped (#31, #179, #181), so it was not sacrificed; UC9 admin console: frontend ADR 0015, admin stacks #294–#338.
- Money path on dev: #517 (16 Oct, receipt stayed "Waiting for payment confirmation") and #223 (Checkout, scheduling and expiry refunds on dev), owner Yorik. PR #664 (merged 24 Oct, bef39b72) repaired the dev webhook endpoint and turned scheduling on, on the unchanged backend image: a real Sandbox START settled and created its bloc and order, and a natural expiry refunded S$22 once, 40.6 s after the deadline (backend/docs/payment/stripe-dev-recovery-acceptance-20261023.md). A second buyer's paid JOIN was shown on dev on 24 Oct (system demo backup recording, #657: "2 of 2 joined · Filled 24 Oct, 7:17 pm"). Not shown on dev: crash recovery under replay (#224). Yorik closed #517, #223 and #590 on 25 Oct (01:40 SGT) on this dev evidence: dev only, no production readiness, the #590 JOIN is a recorded UI observation (no database or log audit), and crash and replay stay in #224 (open).
- DevSecOps row (proposal §5.2, §6.4): OWASP Dependency-Check promised as a blocking check is not used; Trivy deps (Maven, pnpm) and Trivy config (IaC, Docker) run on PRs but are not required checks (ruleset 22734681 requires build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend); the three plan/deploy/apply roles became three per-job roles (ECR push, backend deploy, frontend deploy); "no static credentials anywhere" is not met: two IAM users for SSM port forwarding, one with an active key (docs/deploy/aws-account-governance/README.md). No ADR for any of these (open item 7).
- Changed: team lifecycle State → DealStatus.canTransitionTo transition table (backend/docs/design/paid-group-participation.md; State is used for listings, slide 23). JWT → Sa-Token session in Redis with an HttpOnly cookie (backend-skeleton-plan.md D6/D7; frontend ADR 0002; no backend ADR, open item 7). JPA → MyBatis behind repository ports (skeleton plan §3 stack; no ADR, open item 7). ECS Fargate + RDS for dev → one EC2 host with Docker Compose and PostgreSQL on the host (backend ADR 0008). Secrets Manager → SSM SecureString and a root-only host env file (docs/deploy/backend-ec2-pipeline-operator-guide.md §7). UC9 "listing administration is API-only CRUD plus DB seeding" (proposal §6.1, §6.2: no merchant self-service GUI) → an admin console for admins (frontend ADR 0015, PRs #305–#307, 30 Sep). Proposal §6.4 patterns: Adapter for the Stripe payment abstraction → gateway ports under the external-interface transition strategy (slide 10; the GoF patterns are on slides 18–23); Observer for team resolution → kept, presented on UC5 → UC6 (slide 19). No ADR for the pattern changes (open item 7).
- Dropped for SWE5006: the prod release with a human gate and the ECS circuit breaker (#226 closed not planned 11 Oct; both exist for SWE5001 on ECS). Partial: Playwright e2e runs on API stubs, the signed-in suite on dev (#229) was not planned; burndown data is captured daily (board-snapshot.yml since 17 Sep) but no chart is generated;

saved-plan Terraform promotion is a manual operator step only. Delivered: Checkstyle and both SonarCloud gates as required checks.

- Open gaps: port 22 open to 0.0.0.0/0 on the dev host (AWS read-only, 21 Oct); CloudFront → ALB over HTTP accepted until 16 Nov (#227, NFR-37); dev EBS volume unencrypted (AWS, 19 Oct).
- If asked "microservices or monolith?": BlocBuy is a modular monolith (COLA modules). SWE5006 dev runs it as one service on one EC2 host; for SWE5001 the same code is deployed as three services on ECS, and that is where the gated production release and circuit-breaker rollback exist (#226).

Hand-over *"How we got here, in milestones and hours."*

Slide 5 Milestones achieved and effort per member

16:31:30–16:31:50 · 20 s

Speaker Lim · covers guideline item **1d**

Say *On the left, we have milestones: we have a tag per sprint, pipeline on dev in week one, the web app on 22 September, production release on 20 October. On the right: planned hours, and actual estimated man-days so far.*

≈ 40 words · ≈ 18 s at 130 wpm · slot 20 s · if short of time: If over time, drop "a tag per sprint".

Cues

- Milestones: proposal header (27 Aug); Sprint 1 report (dev pipeline live 5 Sep); stripe-sandbox-acceptance-20260920.md (local Sandbox acceptance 20 Sep); #93 comment (web app on dev 22 Sep); tags v0.1.0-sprint1/2/3 (14 Sep, 27 Sep, 11 Oct; git tag); admin console PRs #305–#307 merged 30 Sep (gh pr view, 22 Oct). Not milestones, so not listed: the 30 Sep local performance baseline and PR #424 (13 Oct).
- Planned hours: Sprint 2 plan docx (30 h each), sprint-03/plan.md §4 (30 h each), sprint-04/plan.md §5 (Liu Jin 51, Yorik 48, Eric 46, Pan 41.5, Lim 38). Open PR #538 rebalances Sprint 4 to Liu Jin 66, Eric 26, Pan 46.5 (open on 21 Oct). Sprint 1 hours were not recorded (Sprint 1 report contribution table left TODO). Team done hours: Sprint 2 134 of 150 (Sprint 2 report), Sprint 3 117 of 150 (sprint-03/review.md).
- Actual (person-days): Liu Jin 12.5, Pan 11.5, Yorik 9, Lim 8.5, Eric 8.5, total ≈ 50; the same figures as the SWE5001 talk on 21 Oct (docs/swe5001/presentation/src/slides-main.js eff, Q&A in its content.js: “we added up the planned hours of the issues each person actually delivered, and the team agreed the final split”). One project serves both courses, so one effort split. Within the proposal budget of 10 person-days each (proposal §7) and the SWE5001 course guideline of 8.5–12.5. Rubric 1.4: the fortnightly progress reports carry no hours per member (Sprint 1 TODO, Sprint 2 activities and PR counts), so the planned hours are never presented as actuals; the actuals are this agreed estimate, not logged time.

Hand-over *"Who owned what."*

Slide 6 Ownership: at least one use case and one design problem each

16:31:50–16:32:00 · 10 s

Speaker Lim · covers guideline item **1d**

Say *Every teammate owns at least one use case. We'll present three of those later.*

≈ 14 words · ≈ 6 s at 130 wpm · slot 10 s

Cues

- Ownership: issue #463 and sprint-04/plan.md §11 for the use cases; design problems as decided on 21 Oct (Strategy for UC1 pricing assigned to Lim, Template Method to Pan, Observer to Yorik). Proposal §6.1 indicative allocation: UC1 Strategy Pan; UC2 Pan; UC3 Factory Method and UC4 Eric; UC5 State/Observer and UC9 Liu Jin; UC6 Adapter Yorik; UC7 Observer and UC8 Jiechao.
- Owner strip (all nine use cases, rubric 1.5), decided 24 Oct by the report editor (#463 comment): UC1, UC2 Pan (as in the proposal); UC3, UC4 and the included pay upfront Eric; UC5 Liu Jin (as in the proposal, and he built it, Sprint 2 report); UC6 Yorik; UC7, UC8 Lim (as in the proposal); UC9 Liu Jin, with Eric credited for the first version (PR #72, 11 Sep). Notifications are SWE5001 scope (use-case diagram: SQS consumer runs on SWE5001 only); Pan may keep them as extra material. The template requires each member to own at least one use case and every in-scope use case to have its four diagrams (RT 3.4). Presentation is unchanged: anyone presents.
- Also on the matrix: the team-formation statechart (UC4–UC6, slide 17) is presented by Eric; in the report it sits in Liu Jin's UC5, §3.4.5 (the code, DealStatus and the GroupDeal guards, serves UC3–UC5).
- Report sections: 3.4.N (use case by member, RT 3.4) and 3.5.N (design problem by member, RT 3.5) follow the deck's section 2 presenting order: 1 Eric, 2 Yorik, 3 Pan, 4 Lim, 5 Liu Jin (#463 report plan, 24 Oct; sub-issues #678, #679); the use-case descriptions are §3.3.2–§3.3.12 (#677). The order is the speaking order of slides 9–24, so it did not move when open decision 4 was settled. Settled by the drafts merged 23 Oct (docs/swe5006/report-drafts/): §3.2 transition-strategies.md; §3.4.1 Eric eric-wang-contribution.md (UC3, UC4) and statecharts.md (team statechart); §3.5.1 Eric the Factory Method section of eric-wang-contribution.md.

Hand-over *"And how each sprint went."*

Slide 7 Four sprints: the Definition of Done reached, and what each retro changed 16:32:00–16:32:35 · 35 s

Speaker Lim · covers guideline item **1e**

Say *Each row places a sprint on the EPAT Done ladder. For three sprints we stalled at integrated: merged and tested, but never shown working end to end on deployed dev. Sprint 4 got past it: the same code was released to production through our SWE5001 cloud setup, and on dev it's releasable.* **[click 1/1]** *Each retro led to a change: no burndown became daily board snapshots; a ten-thousand-line pull request became a 1,500-line limit.*

≈ 72 words · ≈ 33 s at 130 wpm · slot 35 s · if short of time: If over time, drop "and a mid-sprint check".

Cues

- Sprint dates, one set: docs/agile/definition-of-done.md “Done ladder per sprint”: Sprint 1 1–12 Sep, Sprint 2 15–26 Sep, Sprint 3 28 Sep–9 Oct, Sprint 4 12–23 Oct; dates follow the progress reports where they exist (Sprint 2: docs/sprint2-progress-report.typ L81, L118, the same period as the burndown on slide 8).
- Ladder: EPAT 01 Agile p26 (coded → tested → integrated → releasable → released). Placing each sprint on it is our own reading of the evidence, not a course rating. Basis, the same for every sprint: the sprint goal’s increment; integrated = merged to dev with all required checks green, releasable = demonstrated on deployed dev (sprint-03/plan.md §8 “on dev” rule), released = promoted to production through the gated release path and verified there by the smoke suite (SWE5001 only; Stripe test mode; the ladder asks for a release, not for adoption by outside users). The footnote’s basis and the four placements are on one page: docs/agile/definition-of-done.md#done-ladder-per-sprint (current list: #current-definition-of-done-sprint-4).
- Sprint 1 (1–12 Sep, #49): backend pipeline live on dev 5 Sep with health check and rollback; frontend not started (Sprint 1 report). Sprint 2 (15–26 Sep, #93): 9 of 11 stories, 134 of 150 h; frontend live 22 Sep; Checkout and scheduling off on dev (Sprint 2 report). Sprint 3 (28 Sep–9 Oct, #232): 117 of 150 h; review records the goal as not met on dev (sprint-03/review.md); DoD rule "on dev means demonstrated on deployed dev" (sprint-03/plan.md §8). Sprint 4 (12–23 Oct, #465): released on SWE5001 production: three-service release first promoted by PR #534 (17 Oct; an earlier monolith release, PR #418, ran on 6 Oct), last by promotion PR #574 merged 20 Oct, deploy run 39435280952 green with the smoke suite passing against the production CloudFront at 14:31 UTC, release digests in SSM /blocbuy/swe5001/release/* (AWS read-only, 23 Oct); the SWE5001 runtime is brought up for sessions and torn down after (infra/swe5001/README.md; no ECS cluster on 23 Oct), and served the 20–21 Oct sessions including the SWE5001 talk. Stripe stays in test mode there. SWE5006 dev: releasable: paid START and the expiry refund on deployed dev on 23 Oct (PR #664, merged 24 Oct, bef39b72), and a second buyer’s paid JOIN shown on dev on 24 Oct (system demo backup recording, #657: “2 of 2 joined · Filled 24 Oct, 7:17 pm”); #590 closed on that recording 25 Oct. The team agreed this Sprint 4 step and the released reading on 24 Oct (#619). SWE5006 itself has no production (#226).
- Retro → change: Sprint 1 retro (Sprint 1 report: no burndown or story points) → dated day logs and the scheduled board snapshot to docs/agile/metrics/ from 17 Sep (Sprint 2 report, "Acting on the Sprint 1 retrospective"). Sprint 2 retro (Sprint 2 report: ≈ 10,000-line Stripe slice, cliff burndown) → sprint-03/plan.md §8: PRs under ~1,500 lines, remaining hours in the estimate field, mid-sprint check 2 Oct. Sprint 3 retro: docs/agile/sprint-03/retro.md, reconstructed on 24 Oct from the review, the day logs and the Sprint 4 plan (no retrospective meeting was recorded; #451). The one action Sprint 4 adopted: the board estimate field was left empty and the review had to reconstruct the remaining hours (sprint-04/plan.md §7, L141), so in Sprint 4 each owner updates it daily and the mid-sprint check verifies it (slide 7, row 3 → 4). If asked whether it held: the board snapshots show the Sprint 4 estimates set by 12 Oct and unchanged to 23 Oct

(retro.md). Sprint 4 review and retro: after the talk, so they can use its remarks (#619, decided 24 Oct).
- One or two improvements per sprint follows the EPAT retrospective template (SCRUM_Process_Team01.xlsx, "Focus on 1 - 2 improvements").

Hand-over *"Two artefacts from that process."*

Slide 8 Agile artefacts: a story card, a burndown, a practice

16:32:35–16:33:00 · 25 s

Speaker Lim · covers guideline item **1g**

Say *Here we showcase some agile artefacts: A story card with its acceptance criteria. Sprint 2's burndown: flat, then a cliff. After the retro capped pull-request size, Sprint 3 burns from day one. And an example of agile practice: thanks to failing tests, we caught a bug before we refactored.*

≈ 49 words · ≈ 23 s at 130 wpm · slot 25 s

Cues

- Story card: #221 UC9-ext (US25), owner Liu Jin, six acceptance criteria ticked, merged in F (29 Sep), design problem field names State and Optimistic Offline Lock.
- Burndown, both lines reconstructed because no hours were logged, each on 150 planned h. 2 (15–26 Sep): docs/sprint2-progress-report.typ “Daily burndown data”, each story burnec merge date of the PR that met its acceptance criteria; no committed-story PR merged before Sep (#126, the Stripe slice), so the flat week is real; the week-1 merges (#106 scaffold and were carry-over and enabling work outside the 11 committed stories. Sprint 3 (28 Sep–9 Oc docs/agile/sprint-03/review.md planned hours, each Done story burned on its issue close d SGT (gh, 23 Oct; #130/#134 closed Sat 3 Oct, counted Mon 5 Oct): 132, 114, 90, 82, 76, 6 68, 66, 66; the plateau is #223, #224, #229 waiting for Checkout in dev and #226 for NFR- Whole stories only: 66 h left; the review credits partial work and records 33 h at freeze. Daily counts since 17 Sep: docs/agile/metrics/*.csv; both charts in backup B2.
- Practice (EPAT 05 refactoring, EPAT 03 TDD): two red tests reproduced the below-one-cou quote defect before the fix (docs/agile/sprint-03/log/2026-10-07.md, GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted), then PR (13 Oct) refactored admission into shared guards and a constructor-injected Factory Metho (#421, #422); guard-order and golden-hash tests pin the refactoring (GroupDealPurchaseSupportImplTest.closedAndFullTeamsAreRejectedBeforeDuplicateMem PaidGroupDealTest.admissionGuardsRejectMissingFactsAndPreservePaidRejectionPriority, PaidGroupDealServiceTest.historicalStartAndJoinFingerprintsRemainByteCompatible, #423; wang-contribution.md “Implementation decisions”).
- Pairing (EPAT 02), if asked: #469 / PR #490 (14 Oct, sprint-04/log/2026-10-14.md), Yorik Jin paired on one branch and one PR; the only recorded pairing, and how the pair worked (d and observer) is not recorded (docs/agile/practices.md#pair-programming-epat-02).
- Full set (rubric 1.8: product backlog, user stories, sprint plans, burndowns, reviews, retros): B1, burndowns B2.
- If asked whether the burndown steered the sprint: no. Both lines were rebuilt afterwards fron merge dates (Sprint 2) and issue close dates (Sprint 3) because no hours were logged; that was the Sprint 1 retro finding. What we did track live from 17 Sep was the daily board card c (docs/agile/metrics/*.csv).

Hand-over *"Eric takes you through the design."*

Watch out

Never present planned hours as actual effort (slide 5). Paid start and the expiry refund were shown on dev on 23 Oct (PR #664), a second buyer’s paid join on 24 Oct (system demo backup recording, #657); never

claim crash recovery on dev (#224).

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is Sprint 3 only “integrated” when the code was merged?	Our Definition of Done says "on dev" means shown on the deployed environment; a local run does not count. Checkout and scheduling were still off on dev at the freeze (#223), so the review recorded the goal as not met and carried it into Sprint 4. Evidence: sprint-03/review.md, sprint-03/plan.md §8.	Lim
Two Sprint 2 Done rules are missing from your Sprint 3 list. Why?	They are “implementation matches the approved contract” and “health check passes and rollback remains available”. No decision to drop them is recorded. Both are still in the pipeline: the contract-drift check runs on backend and contract pull requests, though it is not a required check, and the dev deploy health-checks the new container and rolls back where a previous image exists (docs/agile/definition-of-done.md, changes between the sprint lists).	Lim
Are these hours actuals?	The hours are plans. We planned in hours per story; done hours are planned hours of finished work (Sprint 2 report, Sprint 3 review), and the Sprint 4 burndown sums planned estimates of cards not yet Done. Nobody logged time; the actual column is in person-days, the split we agreed from the issues each person delivered.	Lim
Was the effort balanced across the five members?	Within the SWE5001 course guideline of 8.5 to 12.5 person-days each, about 50 in total, the split the team agreed. The plans gave everyone 30 h in Sprints 2 and 3; Sprint 4 started at 38–51 h each. Board cards differ in size, for example one Stripe pull request was about 10,000 lines, so card counts are not effort.	Lim
Why is the effort the same as in your SWE5001 talk?	BlocBuy is one project for both courses: the same repository, board and sprints. We agreed one effort split for the whole project, from the issues each person delivered, and presented it in SWE5001 on 21 Oct; counting it twice would overstate the work.	Lim
The proposal promised JWT. Where is that change recorded?	In the backend skeleton plan of 30 Aug (decisions D6 and D7: a Sa-Token session in Redis, roles from SQL tables) and in frontend ADR 0002 (same-origin /api, HttpOnly cookie). We chose a server session because we can revoke it and the token never reaches JavaScript. There is no numbered backend ADR for it yet (open item 7).	Lim → Liu Jin
Why did you build an admin GUI when the proposal excluded it?	The exclusion was a merchant self-service GUI. Admins needed to configure team terms on real listings to make START possible on dev (US25, #221), and an API-only flow was error-prone in the demo; the console reuses the same versioned API (frontend ADR 0015). It is extra scope, not a replacement.	Lim
What is your biggest open risk for the demo?	A payment step stalling live on dev. Every payment step has been shown on dev: paid START and the expiry refund on 23 Oct (PR #664), a second buyer’s paid JOIN on 24 Oct (system demo backup recording, #657). Each live payment step has its go/no-go and switches to that recording if it stalls. Not shown on dev: crash recovery under replay (#224).	Lim → Yorik

2 Analysis & Design and Design Patterns (5+0 minutes)

Clock	16:33:00 – 16:38:00 (5 min talk + 0 min Q&A; talk ends 16:38:00)
Owner	Wang Ziang (Eric) for 2.1–2.4 and his design problem; then Yu Qianshuo (Yorik), Panpakorn Siripanich (Pan) and Lim Jiechao, each on their own design problem; Liu Jin for the full-form problem and 2.6
Slides	9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

2a — Software architectural decisions that influence the logical architecture.
2b — Transition from analysis to design
2c — Use case diagram
2d — Analysis and design models
2e — Design problems and patterns
2f — Database schemas

Tip

Eric presents 2.1–2.4 and his own design problem; then Yorik, Pan and Lim each have one 20-second slide on their own problem; Liu Jin closes with the full-form State problem and the schema, and carries straight on into §3. The next speaker unmutes during the previous speaker's last sentence.

Slide 9 Logical architecture: COLA layers across six bounded contexts

16:33:00–16:33:32 · 32 s

Speaker Eric · covers guideline item **2a**

Say

Thanks, Lim. This is our logical view, a modular monolith in four layers. The same code also runs as three services on AWS. **[click 1/3]** *First zoom, the controllers. They take the requests and check the login.* **[click 2/3]** *Next zoom, the application services, which run the use cases.* **[click 3/3]** *Last zoom, the domain. It depends on nothing, and seventeen ArchUnit rules keep it that way.*

≈ 61 words · ≈ 28 s at 130 wpm · slot 32 s · if short of time: Drop the words after "Next zoom" for the application services

Cues

- Diagram: src/diagrams/logical-layers.svg: backend Maven modules blocbuy-client, -adapter, -application, -domain, -infrastructure, -bootstrap, plus blocbuy-platform, the SWE5001 service assembly the monolith does not use (backend/pom.xml; report §3.2.1); contexts identity, catalog, groupbuy, order, payment, notification plus common (backend-skeleton-plan.md D1, D4). Invites and credits live inside groupbuy; "share" is only a frontend feature folder. 14 @RestController classes (InternalUserController only in the distributed topology).
- ArchUnit: 17 rules, the @ArchTest static final ArchRule fields of backend/blocbuy-bootstrap/src/test/java/com/blocbuy/architecture/ArchitectureTest.java (grep -c @ArchTest prints 18 only because a comment matches; sprint-04/log/2026-10-13.md “ArchitectureTest 17/17”; docs/agile/practices.md), e.g. domain_is_framework_free, application_uses_ports_not_tech, controllers_never_touch_persistence, no_field_injection, controllers_are_validated.
- Two domain ports are implemented in application, not infrastructure: PaymentBusinessPort (OrderFulfilmentAdapter) and PaymentIntentPort (PurchaseIntentAppService); both are cross-context seams inside commerce. All seams (report §3.2.3, transition-strategies.md “Cross-context seams”): supplier facades CatalogPurchaseSupport, GroupDealPurchaseSupport, GroupDealSettlementSupport; caller-defined ports PaymentIntentPort, PaymentBusinessPort; identity only through UserAppService (ArchUnit commerce_reaches_identity_only_through_user_port).
- Decisions: backend-skeleton-plan.md D1–D20 (30 Aug, pre-ADR locked decisions); backend ADR 0002, 0007 (still marked Proposed in its file although it runs on ECS), 0008; frontend ADR 0001, 0003. Full index: backup B3. Physical view: slide 31.
- Stack versions: Spring Boot 4.1.1, PostgreSQL 18, Redis 8, Sa-Token 1.46; React 19.3.1, TanStack Router 1.170.44, Query 5.104.3, TypeScript 7.0.2 (frontend/package.json, 23 Oct, after #632 and #634).

Slide 10 Transition strategies from analysis to design

16:33:32–16:34:02 · 30 s

Speaker Eric · covers guideline item **2b**

Say

How did our analysis become this design? Through transition strategies. Each row is one design concern and says what the analysis classes turn into. **[click 1/2]** *Next three rows. For persistence, every entity becomes an aggregate behind a repository port.* **[click 2/2]** *And the last three rows. For concurrency, we lock the team's row before adding a member, so we never oversell.*

≈ 58 words · ≈ 27 s at 130 wpm · slot 30 s

Cues

- Template: SAD 04a p36–37 (a strategy deals with one design concern, usually an NFR; impacted object types, use cases, static and dynamic changes). p36 names "concurrency of user requests" as a strategy subject, which is why concurrency is a row.
- Records: backend-skeleton-plan.md D1–D20 (language, persistence D1/D9, distribution D11/D12, security D6/D7); backend ADR 0001 (purchase intent, group terms; lock and unique index), 0002 (expiry transition and events), 0003 (credit after commit), 0004 and 0005 (Stripe path and its gate), 0007 (outbox); frontend ADR 0001 (stack), 0002 (cookie session), 0003 (generated client), 0009 (envelope).
- No ADR: language and framework, persistence (MyBatis, while the proposal said JPA), the scheduling job design, the concurrency approach as such (open item 7; report §3.2 Overview, transition-strategies.md, names the same four). The design docs hold them: backend/docs/design/paid-group-participation.md (UC4 last-seat race, L283–352).
- Security in code: SaTokenConfig global interceptor on /api/v1/** with public paths; StpUtil.checkRole(ADMIN) on admin controllers; X-BlocBuy-Request on money writes (StripeCheckoutController.checkCsrf); RedisRateLimitGateway. Scheduling: TeamExpirySweepJob fixedDelay PT1M under LeasedJobRunner (sys_scheduler_lock).

Slide 11 Typical changes: class and sequence diagrams

16:34:02–16:34:26 · 24 s

Speaker Eric · covers guideline item **2b**

Say *These strategies change our diagrams in two ways. In class diagrams, one analysis class becomes a few design classes.* **[click 1/2]** *Next. In sequence diagrams, a simple call becomes several steps in one transaction.* **[click 2/2]** *And the reason is dependency inversion, so the rules only depend on interfaces.*

≈ 45 words · ≈ 21 s at 130 wpm · slot 24 s

Cues

- Rubric 2.2 asks for typical class-diagram and sequence-diagram changes, each with its rationale; this slide states them in general, slide 12 shows one pair from UC4.
- Class changes: GUI «boundary» → TanStack route, component, Query hook and the generated client (F-ADR 0001, 0003); «control» → @RestController + application service; «entity» → aggregate behind a domain repository port, implemented in infrastructure by a MyBatis mapper (a read port per aggregate root, plus a write-protocol port where one transaction must lock across aggregates: GroupDealRepository for reads; PaidGroupDealRepository runs the paid path in one transaction that locks the payment receipt, then the deal (ADR 0001, 0002), so the webhook and the expiry sweeper cannot deadlock. That knowingly bends one aggregate per transaction, for payment idempotency; report §3.2.4, §3.2.8; DAO strategy SAD 04a p75; RUBRIC 2.2 Ruling updated 24 Oct); external «boundary» → gateway port + adapter (ADR 0001, 0004, 0005).
- Sequence change: SAD 04a p68–69 shows the same move for its View Car example (controller → DAO); ours: service → repository.lockDeal() (SELECT … FOR UPDATE) → aggregate.joinPaid() → guarded UPDATE, in one transaction (PaidGroupDealMapper.xml).
- Rationale: dependency inversion; 17 ArchUnit rules keep the domain framework-free (ArchitectureTest.java).
- Why the sequence changes that way (report §3.2.8, transition-strategies.md “Transaction boundary”): the application service owns one transaction per step; OrderFulfilmentAdapter is Propagation.MANDATORY, so it fails outside one; no provider call runs inside a transaction (StripeCheckoutService re-reads the Checkout session first, then settles in a TransactionTemplate); consequences of the same fact run BEFORE_COMMIT (OrderLifecycleSubscriber, TeamExpiryRefundSubscriber), derived side effects AFTER_COMMIT (ShareCreditSubscriber, OpenDealOfferCacheSubscriber).

Slide 12 One strategy pair: analysis objects become design classes

16:34:26–16:34:46 · 20 s

Speaker Eric · covers guideline item **2b**

Say

One pair, from UC4. **[click 1/3]** *Before: slide 14's analysis classes, with no platform.* **[click 2/3]** *The key says what each strategy makes of each type.* **[click 3/3]** *Now as design classes: the Team becomes an aggregate behind a repository port, which MyBatis implements.*

≈ 38 words · ≈ 18 s at 130 wpm · slot 20 s · if short of time: If over time, drop "which MyBatis implements" (it is in the caption).

Cues

- Diagram: src/diagrams/transition-pair.svg; crops 1 (analysis side: JoinTeamUI, JoinTeamControl, Team, PaymentGateway, Payment, the same names and operations as uc4-analysis-class.svg on slide 14, cut to five classes), 2 (strategy key and analysis → design mapping), 9 (PaidGroupDealRepository ◁ PaidGroupDealRepositoryImpl, persistence dependency inversion). The full design side (17 tagged classes) is the overview; slides 15–16 show the UC4 design model itself. Report §3.2 (transition strategies; draft docs/swe5006/report-drafts/transition-strategies.md, “From the UC4 analysis model to design”).
- Analysis names (JoinTeamUI, JoinTeamControl, PaymentGateway) are analysis-model names, not code: SAD 03a p21 names the control after the use case.
- Design classes: products.$productId.tsx + use-checkout.ts (Pres); generated client from contracts/openapi.json (Dist, F-ADR 0003); StripeCheckoutController → PaymentBrowserService → StripeCheckoutService (Dist, Sec); GroupDeal + PaidGroupDealRepository (domain) ◁ PaidGroupDealRepositoryImpl + PaidGroupDealMapper.xml + BizGroupDealDO + GroupDealConverter (Pers); CheckoutGateway ◁ StripeCheckoutGateway (Ext). Port method names: reserveAndLock, lockDeal, insertJoinedMember.
- Analysis → design mapping, all eight UC4 analysis objects in seven mapping rows (report §3.2, transition-strategies.md “From the UC4 analysis model to design”): JoinTeamUI → route, DealProgress, useCheckout; PaymentGateway → CheckoutGateway ◁ StripeCheckoutGateway + StripeWebhookController; JoinTeamControl → split by step into five services; Team + Membership → aggregate GroupDeal (biz_group_member rows); Order → Order + OrderRepository; Payment → CheckoutPayment; Refund → a biz_refund row with no domain class.
- Sequence change: SAD 04a p68–69 shows the same move for its View Car example (controller → DAO with alt [found]); ours goes through a repository port; the persistence strategy gives each aggregate root one or more (GroupDealRepository for reads, PaidGroupDealRepository for the locked paid path; report §3.2.4; SAD 04a p75).

Slide 13 Use case diagram: in scope, out of scope and status

16:34:46–16:35:16 · 30 s

Speaker Eric · covers guideline item **2c**

Say

Before we go into one use case, here are all nine. Buyers and admins use the system, helped by Stripe, Google and a scheduler. **[click 1/2]** *First zoom, the buyer. The key idea is that buyers pay upfront, both to start a team and to join one.* **[click 2/2]** *Last zoom. If a team does not fill up in time, the money goes back automatically.*

≈ 61 words · ≈ 28 s at 130 wpm · slot 30 s

Cues

- Diagram: src/diagrams/use-case.svg; crops 1 (Buyer, money path) and 2 (resolution: UC5, UC6 extend, Scheduler); the Stripe, out-of-scope and delivery-status zooms were removed on 25 Oct (Lim), so those parts are read from the full view: Stripe is a secondary actor of refund and of paying upfront; the greyed package is what the proposal excluded; the badges show paid start and expiry refund on dev since 23 Oct, join and team resolution since 24 Oct. Use case names: proposal §6.1; out of scope: proposal §6.2 (merchant self-service GUI, analytics, recommendations, automatic team matching, email/SMS/push, native apps, live payments).
- Notation: «include» arrow from base to included, «extend» from extending to base (SAD 01b p41–42); a time-driven Scheduler actor as in SAD 03c p5. Google is a secondary actor of UC2.
- Status badges are our own addition (team ruling): UC1, UC2, UC7, UC8, UC9 on dev; UC3 (paid START), UC6 (expiry refund) and paying upfront on dev since 23 Oct (PR #664: Sandbox START settled, natural expiry closed the bloc, one refund); UC4 and UC5 on dev since 24 Oct: a second buyer's paid JOIN filled a team of two, shown on dev on 24 Oct (system demo backup recording, #657: "2 of 2 joined · Filled 24 Oct, 7:17 pm"); #590 closed on that recording 25 Oct. Receive notifications: coded, off on dev (backend ADR 0007 scope decision, 11 Oct). UC7: the same recording shows the invite banner naming the sharer ("wood kik started this bloc and invited you").

Slide 14 UC4 join: the analysis model

16:35:16–16:35:56 · 40 s

Speaker Eric · covers guideline item **2d**

Say

Now UC4, join a team. Its analysis model only uses boundary, control and entity classes. **[click 1/3]** *First zoom, the entities. The Team holds its price, sizes and deadline, and each membership in it comes from one payment.* **[click 2/3]** *Next zoom, the analysis sequence. We check admission before payment, so nobody is wrongly charged. After paying on Stripe's page, a free seat gives a membership and an order.* **[click 3/3]** *Last zoom. If the team filled up in the meantime, the payment is refunded.*

≈ 79 words · ≈ 36 s at 130 wpm · slot 40 s

Cues

- Diagrams: src/diagrams/uc4-analysis-class.svg (crop 2: Team, Membership, Payment with multiplicities) and uc4-analysis-seq.svg (crops 3 and 4: admit, alt [window open and seat free] «create» Membership / [else] «create» Refund). Conventions: SAD 03a p8 (three stereotypes), p29 (attributes only on entities), p49–50 (actors talk to boundaries only).
- The pre-payment check (lockJoinTerms in code) is drawn as checkAdmission with a break [not admissible] fragment. Order is associated only with Team in the analysis class diagram, for readability; the code also links order to payment and member to order.
- Full UC4 models in the report: §3.4.1 (Eric).

Slide 15 UC4 join: the design class diagram

16:35:56–16:36:25 · 29 s

Speaker Eric · covers guideline item **2d**

Say

This is the design class diagram, where the Team became GroupDeal. **[click 1/3]** *First zoom. Payment only reaches groupbuy through a port, inside the same transaction, so payment and team always commit together.* **[click 2/3]** *Next zoom. GroupDeal only changes through its own methods, which check the rules first.* **[click 3/3]** *Last zoom. MyBatis implements the port and runs the SQL.*

≈ 55 words · ≈ 25 s at 130 wpm · slot 29 s

Cues

- Diagram: src/diagrams/uc4-design-class.svg (crops 5: PaymentBusinessPort ◁ OrderFulfilmentAdapter, the context seam; 7: GroupDeal and «interface» PaidGroupDealRepository; 8: PaidGroupDealRepositoryImpl, PaidGroupDealMapper, BizGroupDealDO). Every class and operation carries file:line in the diagram spec; the complete diagram is backup B14.
- Elisions: PaymentBusinessPort declares five operations (lockQuote, lockForSettlement, fulfil, fulfilmentEvidence, lockExpiredTeam; PaymentBusinessPort.java:13–38); the diagram shows the two the join uses. Two domain ports are implemented in application, not infrastructure: PaymentBusinessPort (OrderFulfilmentAdapter) and PaymentIntentPort (PurchaseIntentAppService).
- Notation: typed signatures, visibility, multiplicities (SAD 01b p14–16); strategy tags trace each design class to slide 10 (SAD 04a p37; RT 3.4.1 "the design complies with the applicable transition strategies"). Report §3.4.1 (Eric; draft eric-wang-contribution.md, “Design class diagram”).

Slide 16 UC4 join: the design sequence, paid join by webhook

16:36:25–16:37:12 · 47 s

Speaker Eric · covers guideline item **2d**

Say

This is the design version of that sequence. After we check Stripe's signature and read the payment back from Stripe, everything happens in the big box, which is one transaction. It first saves the event's ID, so a repeated event is ignored. **[click 1/3]** *First zoom. We lock the team row, so two buyers can never both take the last seat.* **[click 2/3]** *Next zoom. The update only goes through if nothing changed, and the last seat confirms the orders.* **[click 3/3]** *Last zoom. If a check fails, the buyer is owed a refund instead of getting a seat.*

≈ 93 words · ≈ 43 s at 130 wpm · slot 47 s · if short of time: Drop "It first saves the event's ID, so a repeated event is ignored."

Cues

- Diagram: src/diagrams/uc4-design-seq.svg; each crop repeats the lifeline header strip (top 180 px). Crops: 3 (lockForSettlement → lockDeal FOR UPDATE), 5 (guarded UPDATE, MemberJoined, opt TeamSucceeded), 7 (red: BizException, compensate, receipt compensation_required). Every message carries file:line in the spec; the whole sequence is backup B14.
- Lock order on the webhook path: deal (StripeCheckoutService.lockForSettlement → OrderFulfilmentAdapter → settlement.lockTeam → lockDeal FOR UPDATE), then intent, then payment (FOR UPDATE), then the receipt; joinConfirmed locks the deal again inside the sam transaction (already held). ADR 0001 (deal → intent → payment) and ADR 0002 (team → int → order → payment) word the global order differently; both hold for their own path, and the combined order is tracked in issue #663 (open). Guard order in GroupDeal.joinPaid: price → new member → window → seat; the first failure decides the code (PAYMENT_MISMATCH, ALREADY_MEMBER, DEAL_CLOSED, DEAL_FULL).
- Guarded update: PaidGroupDealMapper.xml updateDeal ... WHERE current_size = #{expectedSize} AND status = #{expectedStatus}; biz_group_deal.version is incremented but compared (the payment save does compare version: PaymentMapper.xml). Backstops (repo §3.2.8): uq_biz_group_member_deal_user, uq_biz_group_member_payment_id, uq_biz_group_participation_receipt_payment, uq_biz_order_payment_id, uq_biz_purchase_intent_fulfilled_payment, biz_refund.payment_id UNIQUE, uq_biz_refund_provider_id (partial). A refund obligation carries its own row lease: claimed FO UPDATE SKIP LOCKED with a 2-minute lease_until and version + 1, finish fenced by id and version.
- The critical fragment marks the single database transaction (READ COMMITTED, 10 s timeo UML has no transaction fragment, so we use a critical region and say so. Loser path: receipt compensation_required → requireRefund → biz_refund. Evidence: StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPe (local). Report §3.4.1 (Eric; draft eric-wang-contribution.md, “Design sequence diagram — a team (UC4)”).

Slide 17 Team-formation statechart

16:37:12–16:37:22 · 10 s

Speaker Eric · covers guideline item **2d**

Say *The team's statechart.* **[click 1/2]** *OPEN takes paid joins while seats remain;* **[click 2/2]** *the last seat succeeds it, the deadline expires it.*

≈ 19 words · ≈ 9 s at 130 wpm · slot 10 s

Cues

- Diagram: src/diagrams/team-statechart.svg, consistent with DealStatus.canTransitionTo (this == OPEN && target != null && target != OPEN) and the GroupDeal guards. Time event at(expireTime) because the deadline is an absolute time from the terms, not a duration after entering OPEN; TeamExpirySweepJob detects it every minute.
- Why drawn: SAD 03c p4 and p11 call for a statechart when an object's state affects more than one use case; team state gates UC4, UC5, UC6 and UC8. The proposal also promised the state machine (§6.4).
- FAILED and CANCELED are declared in DealStatus but never written (grep, 21 Oct); shown greyed so nobody reading the enum is surprised.
- UML 2.5 details: the initial transition carries only its effect (a confirmed paid START creates the team, in a note), the three paidJoinConfirmed transitions out of OPEN have complete, disjoint guards (admissible and not last seat, admissible and last seat, ¬admissible), and no final state is drawn because SUCCEEDED and EXPIRED still answer late paid joins with compensation_required.
- Report §3.4.1: team, Order and checkout payment statecharts; §3.4.2: refund obligation; the listing statechart goes with State in §3.5.5 (draft docs/swe5006/report-drafts/statecharts.md); the report's team chart leaves FAILED and CANCELED out instead of greying them.

Slide 18 UC3 start a team: Factory Method

16:37:22–16:38:10 · 48 s

Speaker Eric · covers guideline item **2e**

Say *Creating a team is UC3. Before PR 424, the service built its creator itself with new, so every new kind of team meant editing the service. So we redesigned it.* **[click 1/3]** *First zoom. With Factory Method, the service only knows the abstract Creator, whose final initiate runs the start checks first.* **[click 2/3]** *Next zoom. A concrete creator builds the actual team.* **[click 3/3]** *Last zoom. Now a new kind of team only needs its own creator and team class, and the checks stay in one place, at the cost of one more class.*

≈ 89 words · ≈ 41 s at 130 wpm · slot 48 s · if short of time: Drop ", at the cost of one more class"

Cues

- History: the Creator hierarchy exists since b6f7fafd (paid participation). Until PR #424 (merged 13 Oct, 081047f5) the Client still named the ConcreteCreator: `private final PaidGroupDealCreator creator = new StandardPaidGroupDealCreator();` (git show 081047f5^); #424 made it constructor-injected and added GroupDealCreationConfig (#421), so the Client depends only on the abstract Creator. Cause for redesign (SDP 05a p15): creating an object by specifying a class explicitly; without the pattern the service would construct the team itself, which means making GroupDeal.startPaid public (it is package-private, GroupDeal.java:122).
- Polymorphic Factory Method: PaidGroupDealCreator is abstract with a public final initiate and a protected abstract createGroupDeal; StandardPaidGroupDealCreator overrides it. SDP 01a p32–35 presents the parameterised create() as a Factory Method variant and notes on p34 that making it static "forbids subclassing of the base factory class"; ours is the subclassing form. Calling a static create(type) a simple factory rather than GoF Factory Method is our team ruling (rubric 2.5), not the deck's wording. GroupDeal.startPaid is package-private and static, and is not claimed as a pattern.
- What varies (SDP 05a p13, GoF p30): "subclass of object that is instantiated". The pattern provides that variation point: a second ConcreteCreator decides which team class initiate builds, while initiate keeps the start rules (GoF p136: a template method calling a factory method). Needed because: proposal §6.3 makes extensibility the primary design driver (new team-resolution behaviour without editing domain logic). Liability, honestly: the flexibility is not exercised yet; GroupDeal is public final (GroupDeal.java:19) and StandardPaidGroupDealCreator is the only, final, ConcreteCreator, chosen in GroupDealCreationConfig. A second team type would need GroupDeal opened up or a new product class.
- Rejected alternatives (report §3.5.1, eric-wang-contribution.md "Candidate patterns"): direct startPaid (package-private, widens team construction); naming the ConcreteCreator in the service (the pre-#424 cause); static simple factory with a switch (no override, SDP 01a p34); Abstract Factory (one product, not a family); Builder or a static factory on GroupDeal (terms already complete, no override point); creator registry (one team class; #421 excludes it).
- Implementing classes and tests (G10, IDE open): PaidGroupDealCreator, StandardPaidGroupDealCreator, GroupDeal, PaidGroupDealService, GroupDealCreationConfig; PaidGroupDealServiceTest.startUsesTheInjectedCreator, PaidGroupDealTest.createsOpenDealWithExactlyOnePaidLeader, initiationRejectsExpiredTermsInvalidSizeAndMissingClock.
- Diagram: src/diagrams/fm-start.svg (crop A1). Full write-up: backend/docs/design/paid-group-participation.md L108–262; report §3.5.1 (Eric; draft eric-wang-contribution.md, "Design problem: creating the team without naming its class").

Hand-over *"Yorik, what happens when a team ends?"*

Slide 19 UC5 to UC6 team outcome: Observer

16:38:10–16:38:40 · 30 s

Speaker Yorik · covers guideline item **2e**

Say *I'm Yorik. Direct calls for order cancellation and refunds cause tight coupling.* **[click 1/2]** *Let's zoom in on the first diagram. With Observer, Spring's publisher is the Subject. Spring adapts subscriber methods as Observers.* **[click 2/2]** *Now, let's move to the next zoom, the sequence diagram. They run in order before commit. A new action needs a new subscriber. Let's zoom back out. Next, Pan will explain notifications.*

≈ 64 words · ≈ 30 s at 130 wpm · slot 30 s

Cues

- Participants: Spring's ApplicationEventPublisher (notify) and ApplicationEventMulticaster (at observer list) play the Subject, like GoF's ChangeManager; Observer = ApplicationListener; ConcreteObserver = Spring's TransactionalApplicationListenerMethodAdapter, which TransactionalEventListenerFactory creates for each @TransactionalEventListener method ar which realises ApplicationListener (spring-tx 7.0.9); our subscriber classes implement no in (OrderLifecycleSubscriber.java:32, TeamExpiryRefundSubscriber.java:26), the adapter invol their annotated methods; GroupDealExpiryService (and PaidGroupDealService for TeamSucceeded, MemberJoined) is the event source, a client of the Subject, not a ConcreteSubject: it holds no observer list; deal.expire(now) is a call on the GroupDeal aggre note in the sequence); adapted subscribers OrderLifecycleSubscriber @Order(10) and TeamExpiryRefundSubscriber @Order(20), both BEFORE_COMMIT; ShareCreditSubscriber handles MemberJoined AFTER_COMMIT (ADR 0003).
- Why before commit: ADR 0002, both writes commit with the expiry or neither does. Tests (G TeamExpiryRefundSubscriberTest.expiryRecordsRefundObligationsForThatTeam, aFailureToRecordObligationsEscapesAndRollsTheExpiryBack; OrderLifecycleSubscriberTest.expiryCancelsPendingOrdersStampedWithTheTeamsResolutic No test asserts the 10 → 20 order directly.
- Refund idempotency (durable obligation, provider idempotency key, lease and version fence mechanism of the external-interface strategy, not part of the GoF pattern: backup B6. Diagr src/diagrams/observer-expiry.svg (crop A2). Report §3.5.2 (Yorik).
- Liability: implicit control flow, and a wrong phase annotation is a silent bug (ADR 0002 calls i easiest thing here to get wrong").

Slide 20 Notifications: Template Method for outbox transports

16:38:40–16:39:00 · 20 s

Speaker Pan · covers guideline item **2e**

Say *I'm Pan. Our outbox relay publishes to SQS locally and SNS on ECS with the same batching; only two steps differ.* **[click 1/2]** *Template Method: a final publish runs the loop* **[click 2/2]** *and calls entry and send, which each transport implements.*

≈ 38 words · ≈ 18 s at 130 wpm · slot 20 s

Cues

- Participants: AbstractClass BatchingOutboxEventTransport<E> (package-private; public final publish = template method; protected abstract entry and send = primitive operations); ConcreteClasses SnsOutboxEventTransport, SqsOutboxEventTransport; client OutboxRelayService through the domain port OutboxEventTransport; one bean chosen by blocbuy.events.transport (OutboxTransportConfiguration).
- What varies: "steps of an algorithm" (SDP 05a p12–14); Strategy rejected because the whole algorithm does not vary. The brute force (two copies of the loop) is hypothetical: both classes were born in the PR #485 follow-up, with the shared batch publisher (ADR 0007 review amendment).
- Tests (G10): OutboxBatchFailureTest (6, parameterised over both providers: middleBatchExceptionPreservesSuccessesAndStillSendsLaterRows, leaseLossDuringFirstSendSkipsAllSubsequentBatches, ...), SnsOutboxEventTransportTest, SqsOutboxEventTransportTest. Runs locally and in CI against ElasticMQ; the relay is off on SWE5006 dev (ADR 0007 scope). Report §3.5.3 (Pan).

Hand-over *"Lim, pricing."*

Slide 21 UC1 pricing: Strategy

16:39:00–16:39:35 · 35 s

Speaker Lim · covers guideline item **2e**

Say

Pricing use case: Using the strategy pattern, each product has a pricing mode, and that mode decides its price, both on the product listing and in a team quote. Without it, it would have to be hard-coded. **[click 1/2]** *So both services ask a resolver for the mode's strategy* **[click 2/2]** *and hand it the decision. We start off with one strategy; when adding new pricing schemes, as our proposal promised, existing pricing logic would not be touched.*

≈ 74 words · ≈ 34 s at 130 wpm · slot 35 s

Cues

- History: Strategy was there from the first commit (2c6ea4f3, 8 Sep, product browse with dual-price display), so the inline version is hypothetical, as on slide 20; never say pricing "used to" be inline. The pricing is a rule, not arithmetic: DualPricingStrategy.resolve keeps the list price and takes the deal price only if the currencies match; quoteTeamPrice checks the policy and returns its amount. If asked why Strategy for two fixed prices: a new pricing mode changes both the listing price and the team quote, and the services stay untouched.
- Participants: Strategy PricingStrategy (domain, stateless); ConcreteStrategy DualPricingStrategy; Contexts ProductAppServiceImpl, TeamPriceAppServiceImpl; PricingStrategyResolver is a registry (not a GoF participant) built once from all PricingStrategy beans, stores Map.copyOf, fails startup on a duplicate mode and fails a request on an unknown one; key PricingMode (only DUAL), from biz_product.pricing_mode.
- Liability (SDP 05a p35, "apply only if the flexibility is needed"): one ConcreteStrategy today (backend/docs/design/team-price-quotes.md L55). Justification: proposal §6.3 names "new pricing schemes ... addable without modifying existing domain logic" as a primary design driver; a new mode still needs an enum constant, a class, a bean and tests.
- Tests (G10): PricingStrategyResolverTest.resolvesRegisteredStrategyByMode, duplicateModeFailsFastAtStartup, unknownModeIsBusinessErrorNotSilentFallback; DualPricingStrategyTest (9). Ownership: report section owner; the backend code was written by Liu Jin and Eric (team-price-quotes.md L32–33). Report §3.5.4 (Lim).

Slide 22 Design problem: listing rules that depend on its state

16:39:35–16:39:57 · 22 s

Speaker Liu Jin · covers guideline item **2e**

Say *I'm Liu Jin. An admin can publish, delist, revise or delete a listing, but only in some states.* **[click 1/2]** *First zoom. Brute force repeats a status switch in all four actions.* **[click 2/2]** *Last zoom. The sequence has the same switch. What varies is the state.*

≈ 43 words · ≈ 20 s at 130 wpm · slot 22 s · if short of time: Drop "The sequence has the same switch."

Cues

- Diagram: src/diagrams/state-before.svg. It is the brute-force alternative that backend/docs/design/admin-listings.md L67–68 rejects ("repeated status switches in every caller"), drawn as RT 3.5 allows ("using brute force design if necessary"). It never existed as code; the rules inside the switches are the real ones (ProductStatus.java L12–74).
- Real operation names: Product.publish(), delist(), revise(details), requireDeletable(); client AdminListingAppServiceImpl; endpoint PUT /api/v1/admin/products/{productId}/status (AdminListingController).
- Cause for redesign: GoF State applicability, "operations have large, multipart conditional statements that depend on the object's state". None of the causes on SDP 05a p15 names State, so we do not claim one. What varies: SDP 05a p13, "States of an object".
- Authorship, if asked: admin-listings.md names Eric as implementer and Liu Jin as reviewer; the report owner is Liu Jin, decided 24 Oct (#463). Answer: "I own this design and its report section; the first version was built with Eric in PR #72."

Slide 23 After: the State pattern

16:39:57–16:40:16 · 19 s

Speaker Liu Jin · covers guideline item **2e**

Say *We chose State, because the state decides, not a caller.* **[click 1/2]** *First zoom. Product hands each action to ProductStatus. A state allows its own moves and refuses the rest with 409.* **[click 2/2]** *Last zoom. No switch is left.*

≈ 36 words · ≈ 17 s at 130 wpm · slot 19 s

Cues

- Diagram: src/diagrams/state-after.svg (crops A2, B); the after-sequence alt has one operand per kind of ConcreteState (override: DRAFT, OFF_SALE; inherited default: ON_SALE), chosen by dynamic dispatch, not by a switch; listing statechart DRAFT -publish→ ON_SALE -delist→ OFF_SALE -publish→ ON_SALE, delete only in DRAFT: backup B13.
- Enum constant bodies: ProductStatus DRAFT, ON_SALE and OFF_SALE all have bodies; javac compiles each to an anonymous subclass (ProductStatus$1 … $3), so it is one class per state, shared and stateless (GoF State implementation 2). Defaults throw BizException(PRODUCT_STATE_CONFLICT), HTTP 409.
- Why State here: each state changes four operations, not only which moves are legal. DRAFT allows price, currency and stock edits and deletion; ON_SALE and OFF_SALE lock them; publish and delist differ per state. Implementation decisions: Product is immutable, so a transition returns a new revision with version + 1, and updateListing(..., expectedVersion) returns 412 on a stale version (Optimistic Offline Lock, a persistence mechanism, not GoF). Liability beyond the classes per state: AdminListingAppServiceImpl.changeStatus still switches on the target to pick publish or delist, and Product.isOnSale() compares the status for the browse filter.
- Why not State for the team: DealStatus has one move (OPEN → terminal) and its effects are events (slide 19), so a transition table is enough; State would add three classes with no behaviour (SDP 05a p35). Slide 4 records this change: the proposal promised State for the team lifecycle.
- Tests (G10): ProductStatusTest.onSalePublishThrowsStateConflict, offSaleDelistThrowsStateConflict, currencyOnlyChangeIsRejectedAfterPublication; ProductLifecycleTest (7). Report §3.5.5 (Liu Jin).

Slide 24 Database schema: core entities and mapping decisions

16:40:16–16:40:31 · 15 s

Speaker Liu Jin · covers guideline item **2f**

Say *Now the schema.* **[click 1/2]** *First zoom. A unique index stops a buyer from joining one team twice.* **[click 2/2]** *Last zoom. Another one stops a second refund. We use no foreign keys.*

≈ 29 words · ≈ 13 s at 130 wpm · slot 15 s

Cues

- Diagram: src/diagrams/core-erd.svg: biz_product, biz_group_deal, biz_group_member, biz_purchase_intent, biz_payment, biz_order, biz_refund and sys_user, from Flyway V1–V28 (backend/blocbuy-bootstrap/src/main/resources/db/migration). Crops 2 (deal, membership: uq_biz_group_member_deal_user) and 6 (payment, refund: biz_refund_payment_id_key, an inline UNIQUE on payment_id).
- No FK or CHECK constraints is the team's DDL rule (.claude/skills/pgsql-ddl, V5 header), not a course rule; the application keeps integrity (QA: orphan rows). SAD 04a p55 maps associations to FK columns; we keep those columns without the constraint.
- Other details if asked: biz_purchase_intent has no status column (state from fulfilled_payment_id and expired_at); biz_group_deal.version is incremented, not compared; uq_biz_payment_request_key carries the Checkout idempotency key. All 29 tables: backup B4; zoomable copy: open src/diagrams/full-erd.svg in a browser tab.

Watch out

Handover allowance: speech is about 300 s of the 360 s slots, so the four handovers inside §2 (slides 19, 20, 21, 22) get about 15 s each, rehearsed short: the next speaker unmutes and starts with their first line, no greeting beyond their name. Beyond that, segment 2 has no slack: answer design questions in one sentence and park the rest.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why a monolith on dev and services on prod?	Decision: one codebase, two topologies (backend ADR 0008). Reason: SWE5006 grades a complete runnable system, so dev stays the monolith on one EC2 host; SWE5001 needs three services on ECS. The services are thin assemblies of the same modules, and ArchUnit holds the boundaries in the monolith too.	Eric
Is ProductStatus really the State pattern, or an enum switch?	It is State. Each enum constant with a body compiles to its own subclass, overrides only the moves it allows and returns the next state; the default methods refuse with 409. Product calls its state and does not branch on it. Evidence: ProductStatus.java, ProductStatusTest. One switch remains: AdminListingAppServiceImpl.changeStatus picks publish or delist from the requested target. The team uses a transition table (DealStatus.canTransitionTo), which we do not call State.	Liu Jin
How do you stop a double refund?	A durable obligation, then an idempotent call. The obligation is unique per payment and commits with the expiry; the Stripe idempotency key is built from its ID; a lease plus a version fence stops two workers; after 23 hours the worker only searches Stripe, never creates. Evidence: RefundServiceTest.lostResponseRetriesSameDurableKey. Limitation: proven locally with a provider substitute, not on dev (#223). The states, from pending to succeeded, failed or review, are the refund obligation statechart in report §3.4.2 (statecharts.md).	Yorik
Why a fixed 30 s retry and not exponential backoff?	A fixed delay in RefundMapper.xml: the idempotency key makes each retry safe, and the 23-hour cutoff ends automatic retries. There is no exponential backoff or jitter, so we call it a fixed delay.	Yorik
Why a Factory Method with only one creator?	The creator is a seam: initiate is final, so any creator keeps the validation, and constructor injection (#421) lets a test or configuration supply another creator without editing the service. Limitation: one creator, chosen at build time; we do not claim runtime-pluggable creation.	Eric

Strategy with only one strategy: is that not speculative?	It is a liability we state (SDP 05a p35). The seam exists because proposal §6.3 made new pricing schemes a primary design driver; the selection is data-driven (biz_product.pricing_mode) and the two contexts never name the algorithm. A second mode would add a constant, a class, a bean and tests, and leave browse and quote untouched. We do not claim it has been exercised.	Lim
Why not the State pattern for the team, as the proposal said?	Only OPEN has behaviour. SUCCEEDED and EXPIRED are final and refuse every move with the same error. A transition table (DealStatus.canTransitionTo) and the guards in GroupDeal cover that; State would add three classes with no behaviour. Listings differ: each state changes how four operations behave (slide 23). Slide 4 records the change.	Liu Jin → Eric
Is your repository or DAO a design pattern?	Not a GoF one, so it is not on the pattern slides. It is the persistence transition strategy (SAD 04a p64–65, the simple DAO; p75): domain repository interfaces, implemented in infrastructure by MyBatis mappers. GroupDeal has two, on purpose: GroupDealRepository for reads, and PaidGroupDealRepository, a write protocol that runs start, join, settlement and expiry in one transaction, locking the payment receipt and then the deal, so the webhook and the expiry sweeper cannot deadlock (ADR 0001, 0002). That bends one aggregate per transaction, a trade we accept for payment idempotency; its name says repository, but it is really a unit-of-work contract (report §3.2.4, §3.2.8). The same holds for the outbox, idempotency keys and the Optimistic Offline Lock: mechanisms, not GoF patterns.	Eric
How do analysis and design differ in your UC4 models?	Analysis (slide 14) uses only boundary, control and entity, with attributes on entities and no platform (SAD 03a). Design (slide 15) has the real classes with typed signatures, each tagged with the transition strategy that produced it, and the sequence shows the transaction, the row locks and the guarded update. The full set is in report §3.4.1; report §3.2 maps each analysis object to its design elements (JoinTeamControl splits into five services by step; Refund becomes a biz_refund row with no domain class).	Eric
Is there a database per service?	On dev, no: one PostgreSQL database holds every table in schema domain. On ECS, yes: one RDS instance with one database per service. Each database applies the full Flyway history, and each service role is granted only the tables it owns (ADR 0008). Full list: B4.	Liu Jin
Without foreign keys, what stops orphan rows?	The application. Our DDL rule bans foreign keys and CHECK constraints; that is a team rule, not a course rule. Business tables carry a deleted flag, so a delete hides a row instead of breaking its links. Before a listing is deleted, the use case checks every reference: only a draft with no deals, quotes or price policies can go (AdminListingAppServiceImpl.delete, PRODUCT_REFERENCED). Unique indexes stop duplicate rows under races.	Liu Jin
Do notifications work on dev?	No, by decision: the dev host runs no broker and leaves the outbox writer, relay and consumer off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). The path runs on ECS through SNS and SQS, and locally and in CI against ElasticMQ (OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT).	Pan

3 Types of Testing Conducted (3+0 minutes)

Clock	16:38:00 – 16:41:00 (3 min talk + 0 min Q&A; talk ends 16:41:00)
Owner	Liu Jin
Slides	25, 26

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

3a — Types of Testing Conducted
3b — Well structured source code with unit test cases for classes with more complex behaviours.

Tip

Liu Jin continues from slide 24 without a handover.

Slide 25 Testing on the Agile Testing Quadrants

16:38:00–16:38:38 · 38 s

Speaker Liu Jin · covers guideline item **3a**

Say *How do we test it? We use the Agile Testing Quadrants.* **[click 1/4]** *Quadrant one is unit and integration tests. They cover the team state machine, the last-seat race and refund retries.* **[click 2/4]** *Quadrant two is story tests through the real API, and end-to-end tests on dev.* **[click 3/4]** *Quadrant three is manual checks. We did not test with outside users.* **[click 4/4]** *Quadrant four is security scans, which gate the pipeline, and the load test on the next slide.*

≈ 73 words · ≈ 34 s at 130 wpm · slot 38 s · if short of time: Drop "We did not test with outside users." (the Q3 card says it)

Cues

- Model: EPAT 03 Test Driven Development v2.1 p7 (Agile Testing Quadrants: Q1 unit and component, automated; Q2 functional and story tests, automated and manual; Q3 explorator usability, UAT, manual; Q4 performance, load, security, tools) and p5 (levels: unit, integratio system). The level tag on each row is our classification. Placement: repository, SQL and contract integration tests are technology-facing, so Q1 at integration level; Q2 holds only story-level tests that exercise a user story end to end: StripeRolloutAcceptanceIT (8 tests: re HTTP endpoints, scheduler, Flyway schema, webhook signatures; only OAuth and Stripe network calls substituted) and the Playwright e2e specs.
- Counted in the repository on 22 Oct at a1c6ac6e: find backend -name "*Test.java" = 186 (+ ArchitectureTest), 1,329 @Test/@ParameterizedTest; of the 185, 23 boot @SpringBootTe or Testcontainers (grep -l "@SpringBootTest|Testcontainers|@Container|PostgreSQLContainer", 22 Oct at 68325744 so 162 are unit level; "*IT.java" = 49, 45 of them start Testcontainers (postgres:18, redis:8.1 ElasticMQ), 242 annotations; ArchitectureTest.java = 17 rules (grep -c @ArchTest prints 18 because a comment matches); frontend/src "*.test.ts*" = 77; frontend/e2e = 8 specs, 24 te declarations. Per module: backup B7.
- Complex classes (proposal §6.3 "unit tests concentrated on classes with complex behaviou state machine DealStatusTest, PaidGroupDealTest (terminalStatesRejectJoins, lastPaidMemberSucceedsWithoutMutatingOriginalSnapshot); capacity StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPe (Spring Boot with production ports, PostgreSQL in Testcontainers; twenty paid JOINs release together; asserts 2 members, the leader and one joiner, and 19 refund obligations; a *Test b name, integration by level). StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat is not cited: it races through a test fake of the join path; refund idempotency RefundServiceTest.lostResponseRetriesSameDurableKey.
- Caught: (Q1) sprint-03/log/2026-10-07.md, two zero/negative-count red tests reproduce the quote defect (GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted; #422, fi in #424; guard-order and golden-hash tests pin the refactoring: closedAndFullTeamsAreRejectedBeforeDuplicateMembership, admissionGuardsRejectMissingFactsAndPreservePaidRejectionPriority, historicalStartAndJoinFingerprintsRemainByteCompatible); ArchUnit controllers_are_validate required @Validated on InternalUserController (sprint-04/log/2026-10-13.md, #468). (Q2 ci-frontend runs 37102432667 (3 Oct) and 37182996807 (4 Oct): e2e-dev got 502 from /api/actuator/health inside deploy-existing-ec2 of the same commit. (Q3) none logged from the 20 Sep Sandbox run or the 8 Oct cookie check. (Q4) security.yml weekly full-history gitleaks: 24 matches, 20 SHA-256 digests in a contract baseline re-hashed and fingerprinte (PR #580), 4 badge-token findings still open (slide 32; docs/quality/quality-gates.md).
- Where it runs: ci-backend.yml build-quality (mvnw verify: unit, ArchUnit, ITs; required), contract-check (export, git diff --exit-code contracts/, openapi-diff, not required); ci-

frontend.yml frontend-check (Vitest; required), e2e-preview (PRs), e2e-dev (after deploy-dev). Latest local full backend verify: 2,048 passed, 1 opt-in performance skip (sprint-04/log/2026-10-21.md, base 30b75aaa).

- Q3 evidence: backend/docs/payment/stripe-sandbox-acceptance-20260920.md (browse START and JOIN through Hosted Checkout, local, 20 Sep); sprint-03 8 Oct log: Google sign on dev, Chrome DevTools shows blocbuy-token Secure and HttpOnly; hand mutation probes 12–13 Oct logs. No user acceptance testing with outside users; say so if asked.
- Q4 security: types only here (rubric 3.3). Tools and gates are on slide 30: SAST SonarCloud SpotBugs, dependencies Trivy fs + Dependabot, image Trivy, IaC configuration Trivy config, secrets gitleaks. DAST: dast-baseline.yml (#607, merged 22 Oct), a passive OWASP ZAP baseline of selected anonymous dev responses, weekly on Monday, on demand and on PRs that change the scan itself (not on ordinary application PRs); findings advisory, scanner or coverage failures fail that job; no login, payment or active testing. Slide 25's Q4 detail and sli 30 list it, with its findings (0 High, 2 Medium, 4 Low; #610). Last complete run: dev dispatch 40107924516 on 24 Oct (bf5f3072, after the #682 fix in PR #685), green.

Slide 26 Scripted performance test: tool, scenario, target

16:38:38–16:39:08 · 30 s

Speaker Liu Jin · covers guideline item **3a**

Say *That load test is a k6 script. It runs on a cloud test copy, never on the dev host. The load rises from 10 to 200 requests a second. The proposal asks for 2 seconds with 100 users, about 190 requests a second, so our top step covers it. Teams must also close within a minute of their deadline. Yorik shows both results in the demo.*

≈ 66 words · ≈ 30 s at 130 wpm · slot 30 s · if short of time: Drop "Teams must also close within a minute of their deadline." (the expiry card says it)

Cues

- Tool and scenario: perf/k6/README.md §V2 and perf/k6/browse.js: grafana/k6:1.6.1 pinned by digest; ramping-arrival executor, 10, 20, 40, 60, 80, 100, 125, 150, 175, 200 iterations/s, 60 s each; one request per iteration, 7 list : 3 detail; no sleep; 100 preallocated VUs, 400 max; 10 s request timeout; gates per plateau p95 < 500 ms, error rate < 1%, zero dropped iterations (calibrate.mjs). Anonymous browse only: authenticated load needs real sessions.
- Environment: SWE5001 ECS, CloudFront d3lvq02o2e7tcn.cloudfront.net → ALB → ECS Fargate; the README says "Do not load-test shared SWE5006 dev". Say "SWE5001 environment, an image built from dev on 6 Oct" every time (image 6c7a99ab, 38 backend commits before 9c123c12); never call it a dev result.
- Target: proposal §6.3 "Interactive operations respond within 2 seconds under demonstration load (up to 100 concurrent users); team expiry is resolved within 1 minute of window lapse." Little's law N = X(R + Z): N = 100, Z = 0.5 s think, R ≈ 0.03 s gives X ≈ 189 requests/s. The 0.5 s think time is our assumption; with longer think times the offered load is lower.
- History, not the 5.3 result: SWE5001 V2 baseline 16 Oct (docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md), one monolith task, two runs of 57,600 requests, zero errors, every plateau p95 ≤ 57 ms; saturation not observed, so 200 requests/s is a tested lower bound. Local Node workload 30 Sep (docs/perf/sprint3-baseline.md): 100 closed-loop VUs, 200 ms think, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms; 30 expiries, worst observation lag 58.18 s. Local only.
- Expiry on ECS: the V4 burst (2,000 teams, perf/k6/SCALING.md) has not been run, so expiry evidence is local only. TeamExpirySweepJob runs with fixedDelay PT1M, so the bound is one interval plus the sweep time; measured worst lag 58.18 s with 30 teams (docs/perf/sprint3-baseline.md). The 5.3 run is the 16 Oct V2 repeat (run 20261016-v2-monolith-repeat, docs/swe5001/evidence/20261016-http-baseline-dashboard/v2-repeat-summary.json; #670); if a 24 Oct pre-run passes, swap in its numbers.

Hand-over *"Pan, where do all these tests run?"*

Watch out

Never present a local or SWE5001 result as a SWE5006 dev result: say where each number ran.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How do you test payments	Stripe sits behind two ports, the Checkout gateway and the Refund gateway. Unit tests replace the port. StripeRolloutAcceptanceIT (8 tests) runs the real HTTP endpoints, the scheduler, the Flyway schema and the webhook signature checks; only OAuth and the calls to Stripe are replaced. On	Liu Jin → Yorik

without calling Stripe?	dev on 23 Oct, a paid Sandbox START settled and a natural expiry refunded once (PR #664, merged 24 Oct, bef39b72); a second buyer's paid JOIN was shown on dev on 24 Oct (system demo backup recording, #657).	
Why Testcontainers rather than H2?	Our rules live in the database: unique keys, ON CONFLICT DO NOTHING, guarded updates and role grants. H2 accepts different SQL, so we test on real PostgreSQL 18, Redis 8.10 and ElasticMQ in containers. Example: OrderRepositoryIT.duplicatePaymentIsRejectedAtTheDatabase. The cost is Docker and time: the build-quality job takes about 13 minutes (B8).	Liu Jin
Do you do TDD?	For some changes. Four day logs record the red step: new PostgreSQL tests for the offer query failed first (27 Sep, #205); four admin-console changes started from failing tests (1 Oct, PRs #326–#329); the rate-limit tests failed first, 429 expected, 200 returned (3 Oct); two failing tests reproduced the join-quote bug before the fix (7 Oct). Commits are squashed, so the red step lives in the day logs (docs/agile/practices.md). Other changes add their tests in the same pull request, and the 80% new-code gate requires them.	Liu Jin
What is your test coverage?	SonarCloud on dev, read 24 Oct (analyses of bef39b72, the last code change on dev): backend 99.4% and frontend 96.1% overall, both gates passed; on new code 99.3% and 96.2%. The gate needs at least 80% on new code (NFR-06). The frontend figure excludes the generated API client, the route tree and the app entry files (ci-frontend.yml, sonar.coverage.exclusions). Coverage does not measure assertion quality, and we run no mutation testing.	Liu Jin
How do you test the frontend against the API?	Contract first. The backend regenerates contracts/openapi.json, and CI fails if the committed file is out of date; openapi-diff fails a breaking change unless it is labelled. The frontend generates its client and its MSW handlers from that file (frontend ADRs 0003, 0011), and an unhandled request fails a test. The stubs prove only the frontend side.	Liu Jin → Lim
Why is the performance test not on dev?	Dev is one host, and its database runs on the same machine, so a load test would slow down the demo system and its database; the k6 README forbids it. The test runs on the SWE5001 ECS environment, with an image built from dev on 6 October (slide 26; the 16 Oct run is on slide 36). The local Node workload ran on 30 Sep (docs/perf/sprint3-baseline.md). Neither result is from dev, and each is labelled.	Liu Jin
Does CloudFront cache the API, so k6 measures the cache?	No. The /api/* behaviour is uncached, and the k6 preflight checks that before a run (perf/k6/README.md). Every request reaches the ALB and the ECS service.	Liu Jin
Is 200 requests a second the same as 100 concurrent users?	Not directly, so we convert. k6 sends requests at a set rate; the proposal counts users. By Little's law, N = X(R + Z): 100 users who wait 0.5 s and get answers in about 30 ms send about 190 requests a second, below our 200. Our limit is p95 under 500 ms, four times stricter than 2 s. The 0.5 s wait is our assumption. The local run on 30 Sep used exactly 100 users with 200 ms waits, and every p95 was 57 ms or less.	Liu Jin
Is the last-seat race test a unit test?	No, it is an integration test. StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist sends twenty paid buyers through the production intent and fulfilment ports against PostgreSQL 18 in Testcontainers. StripeCheckoutFlowTest.concurrentPaidBuyersCannotBothTakeLastSeat also uses real PostgreSQL, but its provider, group and order parts are test doubles (class javadoc), so we do not cite it. The unit tests of the same rule are PaidGroupDealTest (the aggregate guards) and DealStatusTest (the transition table).	Liu Jin
Why no end-user acceptance testing?	Quadrant three is manual checks by the team, and they are logged: the Stripe Sandbox browser run (20 Sep, local) and the sign-in cookie check on dev (8 Oct). We had no outside users in the course, so we do not claim UAT.	Liu Jin

4 CI/CD Pipeline Demo (10 minutes)

Clock	16:42:00 – 16:52:00 (slides 27–32 until 16:46:30, then the live pipeline demo on slide 33 until 16:52:00)
Owner	Panpakorn Siripanich (Pan); Lim approves as the frontend code owner
Slides	27, 28, 29, 30, 31, 32, 33

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

4a — Describe the strategy to manage the source code and artifacts.
4b — Describe the DevSecOps pipeline and procedures.
4c — Highlight any technical findings and issues encountered.
4d — Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.
4e — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip

Slides 27–32 take 270 s, each slide one topic, each concept explained once. The live demo is frontend only: Pan opens the demo pull request at ≈ 15:30 on the final dev, so its checks (median 4.8 min) finish before the talk. A backend change cannot finish its 16-minute PR checks inside the segment, so the demo shows a backend PR run and its deploy run that finished earlier, said as earlier runs (rubric 4.1).

Slide 27 Source code: one monorepo, Branch by Feature, reviewed changes

16:41:00–16:41:38 · 38 s

Speaker Pan · covers guideline item **4a**

Say *Thanks, Liu Jin. One monorepo holds code, contract, deployment and infrastructure. Our branching is Branch by Feature: one short branch per story, a pull request, a squash merge onto dev. The code owner of each area approves, payments and group buying included, and six checks must pass. People reach AWS through Identity Center, with MFA and one-hour sessions; CI holds no AWS keys and assumes one OIDC role per job. The one exception is two IAM users for port forwarding.*

≈ 80 words · ≈ 37 s at 130 wpm · slot 38 s · if short of time: If over time, drop "payments and group buying included" (it is on the slide).

Cues

- Layout: git ls-files top level (22 Oct). Branching: EPAT 07 Continuous Integration v2.1 p41–44 (Develop on Trunk, Branch for Release, Branch by Feature, Branch by Team). Branch by Feature prerequisites we meet: one branch per story, reviewed merges, short life; the "merge trunk into every branch daily" prerequisite is replaced by the strict up-to-date rule. Linear history: no merge commits on dev since 12 Sep (git log --merges dev). SWE5001 adds a prod release branch; SWE5006 has none.
- Ruleset dev-require-review (gh api repos/PPsyrius/iss-groupbuy-team8/rulesets/22734681, read 22 Oct by the diagram check): 1 approval, require_code_owner_review, dismiss_stale_reviews_on_push, required_review_thread_resolution, strict, six required checks (build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend), squash only. CODEOWNERS: /backend/ @l0sgAi (Liu Jin), /frontend/ and *.js @limjiechao, /.github/ @PPsyrius; payment packages, mappers and docs @Yorick-Yu and groupbuy packages and mappers @EricWang1358, each with @l0sgAi, per Pan's ownership direction of 23 Oct (#650), merged in PR #669 (9a686fcd, 23 Oct).
- People and CI: docs/deploy/aws-account-governance/README.md (Identity Center instance jiechao-aws, one-hour sessions from a permission set, SCP guardrails; IAM users blocbuy-dev-losgai with one active access key and blocbuy-dev-ppsyrius without keys, both BlocBuySsmPortForwarding). OIDC roles: ECR push (trust refs/heads/dev), backend deploy (environment:backend-dev), frontend deploy (environment:frontend-dev), AWS read-only 19 Oct. MFA enforced on Identity Center: Pan's direction at the 23 Oct presentation review (#649); owner-supplied, not re-read from AWS, and the governance README does not state it.

Hand-over *"And what we build from that source."*

Slide 28 Artefacts: each one traceable to a commit

16:41:38–16:42:08 · 30 s

Speaker Pan · covers guideline item **4a**

Say *Every artefact traces to a commit. Images are tagged with the git SHA, and dev runs them by digest. The API contract is committed, and the frontend client is generated from it at build time. Frontend files are content-hashed. Terraform plans are reviewed and applied by an operator, never by CI. Our SBOMs list the source dependencies, not the exact deployed image.*

≈ 62 words · ≈ 29 s at 130 wpm · slot 30 s

Cues

- Artefacts: ci-backend.yml publish-ecr-dev tags git-${github.sha}-run-${run_attempt} and checks the pushed digest with aws ecr describe-images; .github/actions/deploy-ec2/action.yml rejects an image URI not pinned by sha256; deployment/backend/ecr-lifecycle-policy.json keeps 30; ECR immutable (operator guide). contracts/openapi.json + contract-check; frontend ADR 0003 (generated client, gitignored). ci-frontend.yml deploy-s3-dev: hashed assets max-age=31536000,immutable, index.html no-cache. Flyway V1–V28 in blocbuy-bootstrap. infra/terraform/README.md: state in S3 blocbuy-tfstate-402178232802 (encrypted) with DynamoDB lock; "CI never applies it"; the dev EC2 and ALB are outside Terraform. SBOM + licences: sbom.yml (#609, NFR-72) builds CycloneDX 1.7 source inventories with pinned Trivy 0.74.0 of the backend Maven compile and runtime dependencies and the frontend pnpm production closure, plus licence inventories; Actions artefacts, retention-days 90. They are source inventories, not a guaranteed inventory of the exact deployed JAR, image or bundle. Licence findings are report-only; a missing or empty inventory fails the job. Triggers: v* tags, weekly Monday 03:53 UTC, workflow_dispatch, and adding or re-adding the run-sbom label to a PR (a push to an already-labelled PR does not re-run it); ordinary dev merges do not generate one. Evidence: PR runs of 22 Oct at earlier PR heads, 39771223592 (4567873c; the replay baseline in docs/quality/quality-gates.md: backend 195 of 198 third-party artifacts, the 3 optional ones not in the jar; frontend 96 package versions) and 39776023493 (ea95e993). Verified on the merged implementation: run 40076604904, workflow_dispatch on dev 064446f9 (contains PR #609), success, 24 Oct 08:04–08:05 UTC (gh run view); no backend, frontend, contracts or .github file changed from 064446f9 to fba3cfbd (git diff, 24 Oct), so it also covers the current dev revision. The Mon 26 Oct 03:53 UTC scheduled run is a further confirmation, not a precondition.

Hand-over *"Here is what runs on every pull request, and after the merge."*

Slide 29 Continuous integration: pull-request jobs and post-merge jobs

16:42:08–16:42:54 · 46 s

Speaker Pan · covers guideline item **4b**

Say

Here is one change, from pull request to dev. **[click 1/4]** *A backend pull request runs build-quality, with every unit, architecture and integration test, then contract-check for API drift, and docker, which boots the image.* **[click 2/4]** *A frontend pull request runs frontend-check and a Playwright preview, and the security workflow runs on both. Jobs whose files did not change are skipped.* **[click 3/4]** *Then the merge policy you just saw: code owner, six required checks, squash.* **[click 4/4]** *After the merge, delivery rebuilds from the merged commit, deploys to dev and verifies the deploy.*

≈ 87 words · ≈ 40 s at 130 wpm · slot 46 s · if short of time: If over time, drop "Jobs whose files did not change are skipped."

Cues

- Diagram: src/diagrams/cicd-jobs.svg (1164 × 522, shown 1:1; generated in the style of the GitHub Actions run graph after the #605 review: "GitHub CI own ones looks cleaner", "too much text"). Each click highlights one part and greys out the rest. Job names, edges (needs:) and the six required checks (build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend; gh api repos/PPsyrius/iss-groupbuy-team8/rules/branches/dev) checked against origin/dev ci-backend.yml, ci-frontend.yml, security.yml on 23 Oct. detect-changes (the path filter every workflow starts with) and intent-stale are left off the graph. The previous step-by-step diagram is backup F16.
- Off the slide face, for questions (backup F16 shows them): build-quality = Spotless, Checkstyle, mvnw verify (unit, ArchUnit, integration tests), SpotBugs, JaCoCo, the OpenAPI candidate and the verified boot JAR; docker = image ci-<sha>, boot with PostgreSQL 18 + Redis 8.10, /api/actuator/health UP within 60 s, then Trivy; frontend-check = pnpm check (gen:api, tsc, oxlint, oxfmt, Vitest, knip) and build; merge gate = ruleset dev-require-review: 1 approval including the code owner, stale approvals dismissed, threads resolved, branch up to date, squash only; publish-ecr-dev tags git-<sha>-run-<n> and checks the digest in ECR; deploy-existing-ec2 = pre-check, SSM Run Command deploy.sh pulls @sha256, on-host health with automatic rollback, public health via CloudFront; deploy-s3-dev = hashed assets immutable, index.html no-cache, CloudFront invalidation. Other path-filtered PR workflows: typesafe-smoke, perf-tooling, infra-swe5001 (terraform test, Checkov, Trivy config). On a dev push also SonarCloud dev baselines and gitleaks.
- Triggers: pull_request → dev (ci-backend also on labeled/unlabeled, so the contract-change-approved label re-runs the gate); push to dev path-filtered (backend/**, deployment/backend/** with ENABLE_EC2_DEPLOY=true; frontend/**, contracts/** with ENABLE_S3_DEPLOY=true); security.yml weekly Monday 02:17 UTC. On a dev push security.yml runs only gitleaks. Not on ordinary PRs or merges: dast-baseline.yml (passive OWASP ZAP of dev, weekly Monday 03:41 UTC, on dispatch and on PRs that change the scan tooling; advisory, #607) and sbom.yml (SBOM + licence inventories on v* tags, weekly Monday 03:53 UTC, on dispatch and when the run-sbom label is added or re-added to a PR; a push to an already-labelled PR does not re-run it; #609). Jobs skip when their paths are untouched, and a skipped required job counts as passed. The last complete dast-baseline run is dispatch 40107924516 on dev (24 Oct, after #685); the Mon 26 Oct 03:41 UTC scheduled run is a day-of check (slide 30 cue).
- Contract drift: contract-check regenerates contracts/openapi.json from the verified JAR, fails on git diff, and runs openapi-diff --fail-on-incompatible unless the PR carries contract-change-approved; it is not a required check. Policy as code on SWE5006: trivy-config (Dockerfiles, Compose, Terraform, workflows, HIGH/CRITICAL) and deployment-bundle (ShellCheck; Compose may hold only the backend service with host

networking; SSM bundle allowlist and size). Checkov and terraform test run only for infra/swe5001 (infra-swe5001.yml).

- Honest detail (artifact assurance, not an unchecked deploy): on a PR the docker job builds the image from the verified JAR, boots it against PostgreSQL and Redis and scans it with Trivy (required). On a dev push, package-dev rebuilds the JAR with -DskipTests and publish-ecr-dev builds the delivery image again ("Delivery-only: … no post-merge smoke/Trivy", ci-backend.yml), so the deployed image is not promoted from the PR-scanned one and its Trivy image gate is not repeated; the source tree is the same (strict up-to-date + squash). Still after merge: SonarCloud Backend analysis, the on-host health check with rollback, public health and the frontend e2e-dev smoke. ECR scan on push is a separate control, not a blocking gate in this job. Open item 18.
- Timings (gh, 19 Oct, medians of successful runs): backend PR start → docker end 16.1 min; frontend PR → e2e-preview 4.8 min; push → backend deployed 3.1 min; push → frontend e2e-dev green 2.7 min. Table: B8.

Hand-over *"Security sits in each of those stages."*

Slide 30 Security controls by pipeline stage, and what each one catches

16:42:54–16:43:42 · 48 s

Speaker Pan · covers guideline item **4d**

Say

Each control catches something different, at a different point. Before a commit, hooks catch formatting, lint and type errors. On every pull request, gitleaks catches secrets, and the code owner and CodeRabbit catch logic and design problems. In the build, SonarCloud and SpotBugs catch likely bugs, and Trivy and Dependabot catch vulnerable dependencies. Before the merge, Trivy also scans the image and our infrastructure configuration. Weekly, OWASP ZAP scans dev, but only passively and anonymously, and its findings are advisory. One gap: after the merge, delivery rebuilds the backend image instead of promoting the scanned one, and does not repeat that image scan.

≈ 103 words · ≈ 48 s at 130 wpm · slot 48 s · if short of time: If over time, drop "Before a commit, hooks catch formatting, lint and type errors."

Cues

- Stages: DSOEA 07 Secure CI-CD Pipeline v1.0 slide 3 (pre-commit, commit, build, test, deploy/production). Controls: lefthook.yml (oxfmt, oxlint, tsc on staged frontend files); security.yml gitleaks (required; on pull_request gitleaks-action v3 scopes the scan to the PR's commits, security.yml:164; the weekly schedule scans the full history from fetch-depth 0), workflow-lint (actionlint), trivy-deps (Maven + pnpm, HIGH/CRITICAL), trivy-config, deployment-bundle; ci-backend.yml build-quality (SpotBugs, Checkstyle, ArchUnit), SonarCloud Backend and Frontend (required, ≥ 80 % new-code coverage), docker job Trivy image scan (HIGH/CRITICAL, exit-code 1, required). .github/dependabot.yml: daily, cooldown, grouped updates. CodeRabbit (GitHub app) reviews pull requests (drafts on request); the team addresses its findings before merge by practice, for example PR #604; it is advisory, not one of the six required checks. SonarCloud and SpotBugs are the static analysis. Actions pinned to 40-character SHAs (grep of .github, 19 Oct).
- Deploy and runtime: OIDC roles each scoped to one job (ECR push to one repository; SSM SendCommand of AWS-RunShellScript to one instance; one bucket and one distribution); deploy-ec2 rejects a non-digest image URI; deployment/backend/compose.yml read_only, cap_drop ALL, no-new-privileges; secrets in /etc/blocbuy/backend.env root 0600.
- Gaps: DAST only as a passive OWASP ZAP 2.17.0 baseline of dev (dast-baseline.yml, #607; docs/quality/quality-gates.md, NFR-71): selected anonymous GETs and a form-disabled crawl; no active, authenticated or payment testing. Triggers: weekly Monday 03:41 UTC, workflow_dispatch, and PRs that change the scan tooling, not ordinary application PRs. Findings are advisory; a wrong target, incomplete coverage or a scanner failure fails the job; the six required merge checks are unchanged. Verified run: the PR run of 22 Oct (39768665359) recorded 10 of 10 selected responses and 67 of 67 passive rules, 0 High, 2 Medium, 4 Low, all CSP and header settings; the fix is open in #610 (not yet fixed). On dev: dispatch 40076617030 (24 Oct 08:04 UTC) failed as incomplete because the pscanrulesBeta add-on download failed; Pan pinned and checksummed that add-on (#682, PR #685), and dispatch 40107924516 on dev bf5f3072 (24 Oct 15:38–15:40 UTC) passed: 10 of 10 required requests, 67 of 67 passive rules, 0 High, 2 Medium, 4 Low, 6 Informational, the same findings as 22 Oct. Day-of check (run book, ≤ 12:00 row): the Mon 26 Oct 03:41 UTC (11:41 SGT) scheduled run; if it is red, cite 40107924516 as the last complete run; dev delivery rebuilds the backend image without repeating the Trivy image gate (slide 29 cue); four historical gitleaks findings open (PR #580); SSH 22 open on the dev host (AWS, 21 Oct; open item 9). Full list: B9.

Hand-over *"And this is where it all lands."*

Slide 31 Delivery: one environment, the code owner approves, the deploy verifies itself

16:43:42–16:44:39 · 57 s

Speaker Pan · covers guideline item **4b**

Say

Every merge deploys automatically to dev, our only environment. Dev has no environment approval, and it is not a production release. **[click 1/3]** *The browser reaches only CloudFront: files from a private bucket, and API calls to a load balancer that refuses anything without our origin secret.* **[click 2/3]** *One EC2 host runs the container next to PostgreSQL and Redis: cheap and always on, but a single point of failure, with no high availability or auto-scaling.* **[click 3/3]** *A deploy runs through SSM and must pass a health check on the host, a public health check and a smoke test. If the host check fails, it rolls back to the previous image. That rollback exists, but it has never had to run.*

≈ 116 words · ≈ 54 s at 130 wpm · slot 57 s · if short of time: If over time, drop "files from a private bucket, and" and "with no high availability or auto-scaling".

Cues

- Definitions: EPAT 07 Continuous Integration v2.1 p34 ("Deploy to Production is manually triggered" = Continuous Delivery) and p35 (pipeline with Approval). Continuous Deployment automates that production trigger. Dev is not production, so neither term applies to it: say "every merge deploys automatically to dev". SWE5006 has no production (#226), hence no promotion gate.
- Environments: GitHub environments backend-dev and frontend-dev allow only the dev branch and have no required reviewers, so the PR approval is the human gate. SWE5006 prod: infra/terraform/README.md (backend-app-stack plan of 21 creates, not applied, #48), #226 not planned. SWE5001: ruleset prod-release (promotion PR from dev, one CODEOWNERS approval, required promotion-source, service-images, infra-plan); ECS circuit breaker with rollback.
- Diagram: src/diagrams/dev-deployment.svg (UML deployment diagram; full view, then three 1:1 crops). AWS facts: CloudFront E3U5W7UWDL8LRV, S3 behind OAC, ALB default 403 with an X-Origin-Secret forward rule (read-only 19 Oct); EC2 i-0852b6c3475b19f05 t3.large, no ASG, SG 8589 only from the ALB group and 22 open to 0.0.0.0/0 (re-read 21 Oct). PostgreSQL 18 and Redis 8 on 127.0.0.1 (operator guide §1). Not drawn, by decision of 24 Oct: the media CloudFront E1YAIACR5BUHP2 (S3 blocbuy-media-dev, WAF; covers since #570), described in report §4.3 instead.
- Verification: action.yml pre-deploy health → SSM send-command runs deployment/backend/bin/deploy.sh (pull by digest, compose up, on-host health-check.sh at 127.0.0.1:8589; on failure restarts the previous image and the job still fails, deploy.sh:101-141) → public health https://d1b3np88z657cp.cloudfront.net/api/actuator/health; frontend e2e-dev Playwright smoke through CloudFront. Limitation: no automatic rollback has fired yet and rollback-backend.yml has never run (gh, 19 Oct).
- Trade-off (SWE5001 feedback #589, database in its own subnet): on dev the database shares the app host; a deploy restarts the only API (502 on 3–4 Oct). Accepted for a low-cost always-on dev; the SWE5001 topology puts RDS in private subnets (backup B10).
- If asked about production or approvals: SWE5006 has one environment, dev, with no environment approval (gh api repos/PPsyrius/iss-groupbuy-team8/environments, 25 Oct: backend-dev and frontend-dev carry branch policies only, no required reviewers); the code-owner review is a merge gate, not an environment-promotion approval. SWE5006 production was planned but never applied (#226). SWE5001 production on ECS waits for an approved promotion pull request (ruleset prod-release), which the course calls Continuous Delivery; that comparison is for Q&A only.

Hand-over *"Here is what building this taught us."*

Slide 32 Technical findings: three issues, their root causes and fixes

16:44:39–16:45:30 · 51 s

Speaker Pan · covers guideline item **4c**

Say

Three findings. One: early dev deployments failed. The hand-built environment had drifted from its design, with an outdated OIDC subject format and a placeholder instance ID in the deployment policy. We fixed both on dev and wrote the production stack in Terraform; no dev deploy job has failed since 18 September. Two: the build stayed red for two days, because Dependabot upgraded the SpotBugs engine without its plugin. Both now move together and a test guards it, so that is resolved. Three: the weekly secret scan failed on twenty file hashes that were not secrets. We allow-listed exactly those, but four badge-token findings are still open, so that scan still fails.

≈ 111 words · ≈ 51 s at 130 wpm · slot 51 s · if short of time: If over time, drop the third finding from speech; it stays on the slide.

Cues

- (1) docs/deploy/prod/backend-prod-terraform-deployment.md lines 15–23 (dev incident table: OIDC sub format, placeholder instance i-0123456789abcdef0 in the SSM policy, aws CLI missing on the host, 8589 open to 0.0.0.0/0, hand-edited variables); fix commit 121a0733 (immutable subject format). Deploy history, two sources; never call either the complete history or an incident count. (a) Sample of the last 100 runs (gh, 19 Oct; B8): 9 failures 14–17 Sep ("Instances not in a valid state"), 88 of 97 green. (b) docs/ci/dora-metrics-20261022.md (PR #629), all retained attempts, a dev Actions delivery proxy observed through 22 Oct 2026 13:54:34 UTC: 25 failed backend deploy jobs in 5–17 Sep (22 of 47 succeeded), then 103 of 103 from 18 Sep. A job failure is not an incident, publication is not verified service health, and its author-to-dev lead-time proxy is not the ~3-minute push-run duration. Port 8589 now only from the ALB group (AWS, 21 Oct).
- (2) docs/ci/ci-failure-audit-20261021.md: run 39463373272 (20 Oct) missing BCEL PermittedSubclasses with plugin 4.10.4.1 and engine 4.10.5, repeated 19–20 Oct; fixed in PR #580 (commit 0baf2052): backend/pom.xml plugin 4.10.5.0 + engine 4.10.5, Dependabot group spotbugs, .github/tests/backend_quality_test.py.
- (3) Same audit: weekly security.yml gitleaks failed on 21 and 28 Sep, 5, 12 and 19 Oct; 24 generic-api-key findings, 20 of them SHA-256 file digests in scripts/stripe-dev-contract-baseline.json, each re-hashed and matched, then listed by exact fingerprint in .gitleaksignore; 4 Sonar badge-token findings in backend/README.md remain, so the full-history scan still fails.
- Also true, if asked: until 23 Oct the Stripe webhook on dev never confirmed a payment (#517); PR #664 (merged 24 Oct, bef39b72) repaired the dev webhook endpoint and signing secret and turned scheduling on, on the unchanged backend image, and a real Sandbox START and a natural expiry refund were shown on dev (backend/docs/payment/stripe-dev-recovery-acceptance-20261023.md); a second buyer's paid JOIN was shown on dev on 24 Oct (system demo backup recording, #657; #590 closed on it 25 Oct, a recorded UI observation, not a log audit); crash recovery under replay (#224) is not shown; an Actions budget stopped a run on 20 Oct (run 39442654274).

Hand-over *"Now let me show you one change going through."*

Slide 33 Live: one change from pull request to dev 16:45:30–16:51:00 · 330 s

Speaker Pan · covers guideline item **4e**

Say *We'll take a small frontend change through review and deployment, then verify it on dev. While it runs, I'll show a completed backend pipeline. Backend checks take about sixteen minutes, so that run finished before the talk.*

≈ 37 words · ≈ 17 s at 130 wpm · slot 330 s

Cues

- Opens the live pipeline demo: steps P1–P6 in the script, §4 (16:46:30–16:52:00). Frontend only. Pan opens the demo PR at ≈ 15:30, after the 15:00 merge freeze and on the final dev, so its checks (median 4.8 min) finish before the talk; Lim's approval stays pending until P2. Demo change (decided 24 Oct): the home page heading "Buy better together" → "Buy better, together" (one line, frontend/src/routes/index.tsx:114; no test asserts it), author Pan, approver Lim (frontend code owner).
- Rubric 4.1: a full backend run exceeds 8 min (PR checks median 16.1 min), so the demo shows a backend PR run and its dev deploy run that finished before the talk, pinned in tabs 3 and 4 (screen layout, script §4) and said as earlier runs. The frontend change does not build, publish or deploy a backend image. No separate backend demo PR is made; the planned one (#684) merged early as #687.
- Rubric 4.6: backup video swe5006-ci-cd.mp4, recorded on dev 24 Oct (4:18), shared in the team chat: finished ci-frontend and ci-backend runs, a merged PR, the e2e-dev smoke log and the dev site; it shows typical runs, not the live heading change.

Hand-over (demo) "Here is the pull request."

Live pipeline demo (16:46:30 – 16:52:00, 330 s)

Step	Shows	Rubric
P1	The pull request and its required checks	Rubric **4.1**, **4.2** (branch, PR, Conventional Commit, code-owner rule), **4.3** (PR-time jobs)
P2	Code-owner approval	Rubric **4.2**, **4.4** (who approves)
P3	Squash merge	Rubric **4.2** (linear history), **4.4** (the merge is the release)
P4	Delivery runs; a backend PR and deploy that finished earlier	Rubric **4.1** (long run, shown finished), **4.3** (post-merge jobs), **4.4** (digest, SSM, health check)
P5	Verified: the change is live on dev	Rubric **4.4** (how a deployment is verified)
P6	Wrap up and hand over	Rubric **4.1–4.4** summary

Preconditions (checked at 15:30 by Pan)

- Demo branch (frontend only): the home page heading "Buy better together" → "Buy better, together" (one line, frontend/src/routes/index.tsx:114; no test asserts it; decided 24 Oct), author Pan, approver Lim (frontend code owner). After the 15:00 merge freeze, Pan rebases it onto the final `origin/dev`, pushes it and opens the PR at ≈ 15:30; its checks finish by ≈ 15:40. Lim does not approve before P2.
- Merge freeze in the team chat: nobody merges into `dev` from 15:00 to 17:15 except the demo PR, on air. A moved `dev` makes the demo branch out of date and re-runs its checks (strict required checks); if that happens before 16:00, rebase and re-push, after 16:00 leave it and use `CLIP P3`.
- Backend evidence, pinned in tabs 3 and 4: the latest green `ci-backend` pull-request run and the `ci-backend` push run on `dev` that deployed that change, both finished before the talk; they are said as earlier runs, never as this change.
- Latest `ci-frontend` and `ci-backend` runs on `dev` green (A.1); deploy switches `ENABLE_S3_DEPLOY` and `ENABLE_EC2_DEPLOY` unchanged; GitHub Actions budget not exhausted (a budget stop hit run 39442654274

on 20 Oct).

- Lim signed in to GitHub on his own machine with the PR list open; he is not sharing at 16:47.
- Backup video (rubric G6, 4.6): `swe5006-ci-cd.mp4`, recorded on dev on Sat 24 Oct (4 min 18 s): finished `ci-frontend` and `ci-backend` runs with their job graphs, a merged PR, the `e2e-dev` smoke log and the dev site. It shows typical finished runs, not the live heading change, so say so if it is played. Shared in the team chat; open in a player on Pan's machine and tested with Zoom "share sound" off. Never present it as the current live result.

Screen layout (Pan's browser window, shared from 16:46:30; browser zoom 125–150 %, rubric V7)

Tab	Content	Prepared state
1	GitHub: the demo pull request	Opened ≈ 15:30, checks finished; Files changed tab ready
2	GitHub Actions: `ci-frontend` filtered to branch `dev`	Refresh after the merge to see the new run
3	GitHub Actions: a backend PR run that finished earlier	Run summary open; build-quality one click away
4	GitHub Actions: that change's push run on `dev` (deploy)	Run graph open; deploy job one click away
5	Dev site `https://d1b3np88z657cp.cloudfront.net`	On the page the change touches; DevTools closed
Player	Clips P1–P6 (24 Oct)	Paused at the start of clip P1

Step by step

Step P1 The pull request and its required checks — 16:46:30 – 16:47:30

Proves Rubric **4.1**, **4.2** (branch, PR, Conventional Commit, code-owner rule), **4.3** (PR-time jobs) · Pan

Operate

1. Tab 1: the demo PR (opened ≈ 15:30) → **Files changed**: the heading diff in `frontend/src/routes/index.tsx`.
2. Switch to **Checks**: read the run's actual duration aloud; frontend-check, SonarCloud Frontend and gitleaks green; the backend required checks passed by skipping (no backend path changed); e2e-preview green (not required).
3. Point at the merge box: review required from a code owner.

Say *Here is the pull request, opened before the talk: one line, a comma in the home page heading. Its checks finished in [read the duration]. All six required checks show green, but on a frontend-only change the backend ones were skipped by their path filter. Playwright also ran against a preview; it is not a required check. The merge waits for a code owner.*

Expect Checks finished before the talk; merge box asks for a code-owner review.

Fallback A check red: say what it caught in one sentence, then `CLIP P2`; leave the PR open and never merge it on air.

Step P2 Code-owner approval — 16:47:30 – 16:48:00

Proves Rubric **4.2**, **4.4** (who approves) · Pan; Lim approves

Operate

1. Pan posts `APPROVE` in the backchannel. Lim (not sharing) approves from his own GitHub session: **Review changes → Approve**, or `gh pr review <n> --approve`.
2. Tab 1: refresh; the approval from @limjiechao appears; the merge box turns green.

Say *Lim, the frontend code owner, approves now, and the merge box turns green.*

Expect One approving review by a code owner; all checks passed; squash merge enabled.

Fallback Lim cannot approve within 30 s: nobody else is a frontend code owner, so do not ask anyone else. `CLIP P2`.

Step P3 Squash merge

16:48:00 – 16:48:20

Proves Rubric **4.2** (linear history), **4.4** (the merge is the release) · Pan

Operate
1. Tab 1: **Squash and merge**, keep the Conventional Commit title, confirm. Only this PR.
2. Say the merge time aloud.

Say *Squash merge. One commit lands on dev, at [say the time].*

Expect PR merged; leave the branch-delete button alone.

Fallback "Base branch was modified" (dev moved): do not update the branch on air, it would re-run the checks. `CLIP P3`; merge after 17:15.

Step P4 Delivery runs; a backend PR and deploy that finished earlier

16:48:20 – 16:49:50

Proves Rubric **4.1** (long run, shown finished), **4.3** (post-merge jobs), **4.4** (digest, SSM, health check) · Pan

Operate
1. Tab 2: Actions → `ci-frontend` → the new run on `dev`: package-dev → deploy-s3-dev → e2e-dev.
2. Tab 3 (pinned): a backend PR run that finished before the talk: build-quality (test count in the summary), contract-check, docker boot and Trivy, SonarCloud.
3. Tab 4 (pinned): that change's dev deploy run: publish-ecr-dev, deploy-existing-ec2 (SSM, on-host health check, public health).

Say *The merge started frontend delivery: package, upload to S3, then a smoke test through CloudFront. While it runs, here is a backend pull request and its deploy that finished before the talk, because backend checks take about sixteen minutes. This is earlier evidence, not part of this change.*

Expect Frontend run in progress (median 2.7 min from push to e2e-dev green); both pinned backend runs green.

Fallback Frontend run not started within 60 s: refresh once; then `CLIP P4`. Backend tabs missing: open B8 and say the medians.

Step P5 Verified: the change is live on dev

16:49:50 – 16:51:20

Proves Rubric **4.4** (how a deployment is verified) · Pan

Operate
1. Tab 2: e2e-dev green.
2. Tab 5: `https://d1b3np88z657cp.cloudfront.net` on the home page → reload → "Buy better, together".
3. Read the elapsed time off the run: merge → e2e-dev green.

Say *The smoke test is green, and the dev site shows the new heading, [read the minutes] minutes after the merge.*

Expect New text visible after a reload (index.html is served no-cache and invalidated); run green.

Fallback Delivery still running at 16:51:00: say "the deploy is still running", show the job in progress, then `CLIP P5`, said as recorded on 24 Oct. e2e-dev red: show the failed step, it is the gate doing its job, then `CLIP P5`. Text not visible: one more reload, then `CLIP P5`.

Step P6 Wrap up and hand over 16:51:20 – 16:52:00

Proves	Rubric **4.1–4.4** summary · Pan
Operate	1. Stay on tab 5 (the dev site) while speaking. 2. At 16:52:00: stop sharing; Eric starts sharing (`SHARE`) and operates the system demo while Yorik speaks.
Say	*So this change went from an approved pull request to a verified page on dev. The backend runs you saw were earlier runs. Yorik, over to you.*
Expect	Hand-over on time.
Fallback	If behind, say only the last sentence.

Never live

Clicking Merge on anything other than the demo PR; “Update branch” on the demo PR; re-running a failed job; running `rollback-backend.yml` or any `workflow_dispatch`; opening Settings → Secrets and variables or the ruleset editor; pushing to `dev`; any AWS console or SSM session on the shared screen. If the PR is not merged by 16:49, leave it open and merge after 17:15, so a deploy never lands during the system demo.

Tip

Narrate results, not clicks: “checks green in four minutes fifty”, not “now I click the checks tab”. No troubleshooting on air.

Q&A (during the talk / 17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why EC2 and not ECS for dev?	Dev reuses one existing EC2 host with the monolith in Docker Compose (operator guide; backend ADR 0008): SWE5006 needs one complete, always-on system at low cost, and the monolith fits one t3.large. Limitation: one host, so a deploy restarts the API (502 on 3–4 Oct), and PostgreSQL and Redis share the host outside the pipeline.	Pan
How do you roll back?	Automatic: deploy.sh health-checks the new container and restarts the previous digest if it fails; the workflow also checks public health. Manual: rollback-backend.yml (reason, confirmation, same concurrency group). Limitation: rollback-backend.yml has never run and no automatic rollback has fired. Flyway is never reversed, so migrations stay backward-compatible.	Pan
How are secrets handled?	CI holds no AWS keys: OIDC gives short-lived roles per job. The repository has three Actions secrets, none for AWS or the app (SONAR_TOKEN, TYPESAFE_API_KEY, BOARD_SNAPSHOT_TOKEN; gh secret list, 19 Oct). Dev application secrets live in /etc/blocbuy/backend.env (root, 0600). Limitation: the account still has two IAM users for SSM port forwarding, one with an active key (governance README).	Pan
How long from merge to live?	Medians from gh on 19 Oct: backend about 3.1 minutes from the push run starting to the deploy job ending (88 runs); frontend about 2.7 minutes including the e2e-dev smoke (94 runs). PR checks take about 16 minutes for backend and 5 for frontend. Per-workflow table: B8.	Pan
Do you check in to trunk at least daily?	No. EPAT 07 asks for that; we use Branch by Feature instead: one short branch per story, and the ruleset makes every branch be up to date with dev and pass the required checks before it merges. 342 pull requests merged into dev from 14 Sep to 22 Oct, but a daily check-in is not shown (docs/agile/practices.md, continuous integration).	Pan
How do you manage dependencies?	Dependabot checks Maven, npm, Docker and GitHub Actions daily with a cooldown and grouped updates (SpotBugs engine and plugin move together since PR #580); majors of Spring, Java, React and TypeScript are ignored on purpose. Every update passes the same required checks. Trivy scans dependencies on change and weekly. Actions are pinned to commit SHAs.	Pan → Lim
Is the deployed image the one you tested?	Not byte for byte. The squash merge and the strict up-to-date rule make the merged tree the tested tree, but package-dev rebuilds it with -DskipTests and publish-ecr-dev builds the delivery image again without repeating the Trivy image gate; ECR scans it on push, a separate	Pan

	control that does not block this job. SonarCloud still analyses dev after the merge, and the deploy is health-checked and smoke-tested, so this is an artifact-assurance limit, not an unchecked deploy. It falls short of "build once, promote"; frontend ADR 0004 still describes promotion (open item 18).	
Is this Continuous Delivery or Continuous Deployment?	Both terms are about production (EPAT 07 CI p34–35: "Deploy to Production is manually triggered"; automated, it is Continuous Deployment). SWE5006 has no production (#226), so there is no promotion gate: every merge deploys automatically to dev, a non-production environment, after the code owner approves the PR. SWE5001 production is Continuous Delivery: a promotion PR into prod needs a CODEOWNERS approval and three required checks.	Pan
Do you run DAST? And produce an SBOM?	DAST: a passive OWASP ZAP baseline scans dev weekly, on demand and whenever the scan itself changes, not on ordinary application PRs (dast-baseline.yml, #607): selected anonymous pages and a form-disabled crawl, no active, authenticated or payment testing. Findings are advisory, not a merge gate, while a scanner or coverage failure fails the job; the first hosted run on 22 Oct found 0 High, 2 Medium and 4 Low, all CSP and header settings tracked in #610. A dispatch on dev on 24 Oct (run 40107924516) completed with the same findings, after Pan pinned a rule add-on whose download had failed earlier that day (#682). SBOM: sbom.yml generates CycloneDX SBOMs and licence inventories of the backend Maven compile and runtime dependencies and the frontend pnpm production closure, per release tag, weekly, on request and when a PR is given the run-sbom label, not on ordinary merges (#609); a missing or invalid inventory fails the job, while licence findings are advisory. A PR run on an earlier revision listed 195 of 198 backend third-party artifacts (three optional ones, not in the jar) and all 96 frontend production package versions (94 distinct packages); a dispatch on dev on 24 Oct (run 40076604904) passed on the merged revision. It is a source inventory, not an inventory of the exact deployed JAR, image or bundle. Both limits are in B9.	Pan

5 System Demo (12 minutes, then 7 minutes of buffer)

Clock	16:52:00 – 17:04:00 (slide 34 to 16:52:30; live demo on slide 35 to 17:01:00; slides 36–38 to 17:04:00) · 17:04–17:10 buffer for parked questions
Owner	Yu Qianshuo (Yorik); Wang Ziang (Eric) drives the joiner's browser; Lim chairs the buffer
Slides	34, 35, 36, 37, 38, 39

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)
5a — Demonstrate some of the significant use cases of the solution.
5b — Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).
5c — Backup: Record a video in advance in case of any technical issues during the live demo.

Tip
Eric shares one screen at 16:52:00 (SHARE from Pan) with the deck in the Leader window, tab 1, and operates every window; Yorik speaks throughout. The expiry bloc is started at 16:05 so that D4 shows a refund that no one triggered by hand.

Slide 34 System demo: two buyers, one bloc, and the qualities it shows

16:51:00–16:51:30 · 30 s

Speaker Yorik · covers guideline item **5a**

Say

Thanks, Pan. I'm Yorik. I'll present our System Demo. First, we'll look at a completed group purchase on dev. Then we'll show a separate refund case. After that, we'll look at security, the last-seat race, and performance.

≈ 37 words · ≈ 17 s at 130 wpm · slot 30 s · if short of time: If over time, drop "After that, we'll look at security, the last-seat race, and performance."

Cues

- 16:52:00. Eric is sharing one screen with two Chrome profiles, "Leader" (account A) and "Joiner" (account B), and operates both; Yorik narrates (two users: SWE5001 feedback #590). Deck in the Leader window, tab 1, on slide 34.
- Storyline from #464: invite → START/JOIN with Stripe Sandbox → /account orders and blocs → expiry refund. Use cases: UC1 browse, UC2 Google sign-in, UC3 start, UC7 invite, UC4 join (team of two succeeds, UC5), UC5 expiry, UC6 refund, UC8 My blocs. Quality attributes are slide 3's: correctness under concurrency, refund idempotency, expiry ≤ 1 min, security, performance (slide 36), maintainability (§4, slide 38).
- Environment column: every step runs on dev except D6 (a saved CI run) and slide 36 (the SWE5001 ECS run). After the 25 Oct rehearsal (Yorik's script) D1 and D3 show the group purchase both buyers completed in Stripe Sandbox on dev on 24 Oct (product, Checkout form from the recording, saved receipt; the Filled bloc and its confirmed order), D2 plays the teammate's 24 Oct recording of the join and payment, and D4 shows a bloc that expired on dev before the talk; no payment is made during the presentation. This replaces the live START and JOIN that Yorik's GO of 25 Oct (#605) covered. A page that fails to load plays the matching part of the backup recording, said as recorded.

How each step is shown (25 Oct rehearsal; no payment on air)

Step	Shown from	If it fails

D1	The purchase both buyers completed in Stripe Sandbox on dev on 24 Oct: the listing live, the Checkout form from the 24 Oct recording, the Leader's saved receipt live	`CLIP D1`, said as recorded
D2	The teammate's 24 Oct recording of the join and payment (#657), said as recorded on dev on 24 Oct	Say it in one sentence; D3 shows the result
D3	The 24 Oct bloc on dev: "2 of 2 joined · Filled 24 Oct, 7:17 pm", its order Confirmed	`CLIP D3`
D4	The bloc started at 16:05 that expired and refunded on dev before the talk	`CLIP D4`; then B11
D5	Live: one 403 on an admin API	Say what came back; move on
D6	The saved green CI run of the last-seat test	The test file on GitHub

Watch out

Yorik's GO of 25 Oct (#605) covered a live START and JOIN; the 25 Oct rehearsal replaced it: nothing is paid during the presentation. Never start a bloc, open a new Checkout or pay on air. Evidence: START and the expiry refund on dev on 23 Oct (PR #664), the second buyer's paid JOIN on 24 Oct (#657; #590, #517, #223 closed 25 Oct); crash recovery under replay (#224) is not shown.

Preconditions (checked at 15:30 by Eric and Yorik)

- Health `UP` (A.1); latest runs on `dev` green; merge freeze in force; the §4 demo PR is the only change that may land.
- Two Google buyer accounts signed in on Eric's machine in two Chrome profiles, "Leader", shown as **wood kik**, and "Joiner", shown as **los gai** (as in the 24 Oct backup recording); the same two on Yorik's machine as standby. Neither needs admin: the listings check uses the public catalogue, and the Joiner must **not** hold the admin role, or D5's 403 fails; Lim checks it at the 25 Oct rehearsal.
- At **≈ 16:05** (latest 16:07, so the refund lands before the 16:50 check and §5 starts at 16:52) Yorik, as account B, starts a bloc on `US34-04` and pays; nobody joins; receipt bookmarked in the Joiner window, tab 3. It is due ≈ 16:45; the sweep (every minute) expires it and the refund worker (every 30 s) refunds it ≈ 16:46–16:48. Post `EXPIRY OK` at 16:10 and `REFUND OK` / `REFUND LATE` at 16:50.
- Listings converged: `scripts/seed-dev-demo-listings.sh --check` passes for the four `US34-*` listings (≤ 12:00, Yorik).
- D5: the admin URL ready to paste; Joiner DevTools closed.
- D1–D3 sources: the Leader's 24 Oct receipt open in Leader tab 3; the Joiner's `/account` showing the 24 Oct bloc Filled and its order Confirmed in Joiner tab 2; the player cued at D1's Checkout form and at clip D2.
- D6: the saved green `ci-backend` run of the last-seat test pinned in Leader tab 5. The IDE stays open on a local checkout for questions only (rubric G10).
- Backup video (rubric G6, 5.5): `swe5006-system.mp4`, recorded on dev on Sat 24 Oct (8 min 27 s, #657): paid start, a second buyer's paid join ("2 of 2 joined · Filled 24 Oct, 7:17 pm"), My orders and My blocs, the expiry refund ("Refund sent"), the admin API denial, then the code, a CI run and the k6 table. Shared in the team chat; open in a player on Eric's machine (copy on Yorik's), tested with Zoom "share sound" off. Never present it as the current live result.
- Browser zoom 125–150 % (rubric V7); Do Not Disturb on; no password manager, email or bookmarks bar visible.

Screen layout (Eric's shared screen)

Window · tab	Content	Prepared state
Leader · 1	Deck `swe5006-presentation.html#34`	Slide 34; slides 36–39 follow

Leader · 2	Listing `US34-04` on `https://d1b3np88z657cp.cloudfront.net`	Signed in as account A
Leader · 3	The Leader's 24 Oct receipt	"Payment received"
Leader · 5	Saved CI run of the D6 test	Log at the test; test file one click away
Joiner · 2	`/account` (account B)	The 24 Oct bloc Filled; the 16:05 bloc under Refunded
Joiner · 5	Receipt of the 16:05 payment	Bookmarked at 16:10; reload before D4
Player	The 24 Oct recording	Cued at D1's Checkout form, then clip D2

Live system demo (16:52:30 – 17:01:00, 510 s)

Step	Shows	Rubric
D1	A completed start: the first buyer's payment	Rubric **5.1** (UC2, UC1, UC3, UC7), **5.2** security
D2	The second buyer joins and pays	Rubric **5.1** (UC4, UC5, UC7), **5.2** correctness under concurrency
D3	Completed bloc and confirmed orders	Rubric **5.1** (UC8)
D4	Automatic expiry and refund	Rubric **5.1** (UC5, UC6), **5.2** unattended expiry and refund, recorded once (≤ 1 min is measured locally, slide 36, and once on dev, 23 Oct)
D5	Access control	Rubric **5.2** security (slide 3)
D6	Competing for the last place	Rubric **5.2** correctness under concurrency; rubric G10 (IDE on hand)

Slide 35 On dev: two buyers, one bloc

16:51:30–17:00:00 · 510 s

Speaker Yorik · covers guideline item **5c**

Say

The demo follows a buyer's journey through six steps. First, one person starts a bloc and pays the group price. Then a second buyer joins and pays to complete the group. We'll check the confirmed orders and a separate case where the group expires and the buyer gets a refund. Finally, we'll look at access control and how the system handles several buyers competing for the last place.

≈ 68 words · ≈ 31 s at 130 wpm · slot 510 s

Cues

- Opens the system demo: steps D1–D6 in the script, §5 (16:52:30–17:01:00). Eric shares and operates both profiles; Yorik speaks. D1 and D3 show the 24 Oct purchase on dev, D2 its recorded join, D4 the 16:05 bloc, D5 one 403, D6 the saved CI run; no payment on air.
- D1 start (UC2, UC1, UC3, UC7) 120 s · D2 join, team succeeds (UC4, UC5, UC7) 120 s · D3 My blocs and orders (UC8) 75 s · D4 the 16:05 bloc expired and refunded (UC5, UC6; unattended expiry and one refund record; the ≤ 1 min bound is measured locally, slide 36) 90 s · D5 security checks (401, 403, 400, HttpOnly) 60 s · D6 last-seat race, local test in the IDE (correctness under concurrency) 45 s.
- Rubric 5.5: backup video swe5006-system.mp4, recorded on dev 24 Oct (8:27, #657), shared in the team chat: paid start, a second buyer's paid join (2 of 2 joined, filled 24 Oct 7:17 pm), My orders and My blocs, the expiry refund, the admin API denial. With it, D1, D2 and D4 all have dev evidence: D1 and D4 on 23 Oct (PR #664), D2 on 24 Oct (this recording).

Step by step

Step D1 A completed start: the first buyer's payment — 16:52:30 – 16:54:30

Proves Rubric **5.1** (UC2, UC1, UC3, UC7), **5.2** security · Eric operates, Yorik speaks

Operate

1. Leader window (wood kik), tab 2: listing `US34-04` "Universal Travel Adapter" on dev (29.00; team of 2, team price 22.00). Product Page.
2. Player: the 24 Oct recording (#657) paused at the Leader's Stripe Checkout form (Sandbox banner, S$22.00). Never open a new Checkout on air. Stripe Checkout.
3. Leader window, tab 3: the Leader's saved receipt `/payments/{paymentId}` of the 24 Oct purchase, reloaded: "Payment received". First Buyer's Receipt.

Say *First, let's look at our group-buying flow on dev. This is the Universal Travel Adapter. Its single price is twenty-nine dollars. In this example, our first buyer started a bloc for two people and paid the team price of twenty-two dollars. Here is Stripe's checkout page, where the buyer checks the amount and enters their card details. After payment, the buyer returns to BlocBuy to see the result. For this completed purchase, we can see that result on the saved receipt. Here is the first buyer's saved receipt. The buyer paid first and then shared the invite link. We used Stripe Sandbox and a test card, so no real money was charged. BlocBuy confirms payment through Stripe's signed webhook, not just because the browser returns.*

Expect Listing at 29.00 and 22.00; the Checkout form from the recording; the saved receipt shows "Payment received".

Fallback A page does not load within 20 s: reload once, then `CLIP D1`, said as "recorded on dev on 24 Oct".

Step D2 The second buyer joins and pays — 16:54:30 – 16:56:30

Proves Rubric **5.1** (UC4, UC5, UC7), **5.2** correctness under concurrency · Eric operates, Yorik speaks

Operate

1. Player: clip D2 of the 24 Oct recording (#657, the teammate's join), said as recorded on dev on 24 Oct: the invite page with the banner naming wood kik at 1 of 2 (Invitation Page); los gai's Stripe Checkout, S$22.00, Sandbox (Stripe Checkout); the receipt "Payment received · 24 Oct, 7:17 pm" (Payment Received).
2. No payment on air: both buyers paid in Stripe Sandbox on 24 Oct.

Say *This is the second buyer's view. The buyer opens the invite link, and the banner shows who shared it. There is one member out of two, so one place is still available. The second buyer uses a different Google account to join the same bloc. The buyer reaches Stripe's checkout page. The amount is twenty-two dollars, and this is Sandbox mode. The buyer uses a test card and presses Pay. Now the receipt shows Payment received. The system checks the join rules before payment and checks them again after payment under a database row lock. If the last place has already gone, it records a refund instead of adding an extra member.*

Expect The recorded join: sharer banner at 1 of 2, Checkout, "Payment received".

Fallback Player fails: say the step in one sentence and go to D3, which shows the result on dev.

Step D3 Completed bloc and confirmed orders

16:56:30 – 16:57:45

Proves Rubric **5.1** (UC8) · Eric

Operate

1. Joiner window (los gai), tab 2: `/account` on dev: My blocs → **Completed**: the 24 Oct bloc, "2 of 2 joined · Filled 24 Oct, 7:17 pm"; My orders below: its order "Confirmed".

Say *Both buyers have paid, so this bloc shows two out of two and is marked Filled. The matching order below is Confirmed. Payment received tells us that an individual payment succeeded, while Filled tells us that the whole group is complete. Both orders were confirmed in the same transaction, so the two buyers see a consistent result.*

Expect The 24 Oct bloc at 2 of 2, Filled; its order Confirmed.

Fallback Page empty: reload once; then `CLIP D3`.

Step D4 Automatic expiry and refund

16:57:45 – 16:59:15

Proves Rubric **5.1** (UC5, UC6), **5.2** unattended expiry and refund, recorded once (≤ 1 min is measured locally, slide 36, and once on dev, 23 Oct) · Eric operates, Yorik speaks

Operate

1. Joiner window, tab 5: the receipt of the 16:05 payment (bookmarked at 16:10), reloaded: "Refund sent".
2. Joiner window, tab 2, My blocs → **Refunded**: that `US34-04` bloc, "Closed"; its order cancelled. The row shows the closing day, not the time (BlocRow.tsx, formatDay).
3. Never open the Stripe dashboard on the shared screen.

Say *This is a separate purchase where the bloc did not fill before the deadline. The system closed the bloc, cancelled the order, and saved a refund record. Then a background worker sent the refund to Stripe. The receipt now shows Refund sent. If the worker retries, it uses the same idempotency key, so it does not create a second refund.*

Expect Receipt "Refund sent"; bloc under Refunded, "Closed"; order cancelled.

Fallback `REFUND LATE` at 16:50 and still none now: Yorik says "the worker retries every 30 seconds; it may still be in flight", then `CLIP D4`. No dev recording: open **B11** and say "*verified locally with a real Sandbox refund on 20 September.*"

Step D5 Access control

16:59:15 – 17:00:15

Proves Rubric **5.2** security (slide 3) · Eric operates, Yorik speaks

Operate

1. Joiner window, new tab: `https://d1b3np88z657cp.cloudfront.net/api/v1/admin/products` → `403`, `COMMON_403_001` (a buyer fails `StpUtil.checkRole(ADMIN)`, AdminListingController).

Say *Here, a normal buyer is trying to open an admin-only API. The server returns 403, or Permission denied. So being logged in does not give a buyer admin access. The check happens on the server, not just by hiding a button on the page.*

Expect 403, COMMON_403_001.

Fallback Anything other than 403: say what came back in one sentence and move on. The 401, 400 and HttpOnly checks stay for questions (Q&A "Is 401 vs 403 vs 400 deliberate?").

Step D6 Competing for the last place 17:00:15 – 17:01:00

Proves	Rubric **5.2** correctness under concurrency; rubric G10 (IDE on hand) · Eric shows, Yorik speaks
Operate	1. Leader window, tab 5: the saved green `ci-backend` build-quality run (the backend PR run pinned for §4): its log at `StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersis` passed. 2. Same tab, the test file on GitHub at its assertions: 2 members, 2 orders, 19 refund obligation `current_size` 2, stock 98.
Say	*This PostgreSQL test simulates twenty paid buyers competing for one remaining place. The checks expect two members in total, including the original buyer, and nineteen refund records for the others. A row lock lets one transaction update the bloc at a time, so it stays within its limit. These checks passed in our saved CI run.*
Expect	The passed test in the saved run; its assertions visible; then back to the deck.
Fallback	Run tab missing: open the test file on GitHub and its last green CI run from the Actions list.

Never live

Editing any deadline, price or listing on shared dev; any SSM command, database client or AWS console on the shared screen; the Stripe dashboard (keys and webhook secrets live there); the DevTools Application or Network tab with cookies or headers visible; racing two buyers or replaying a webhook on dev; a second payment to “try again”; pointing k6 at dev.

Plan C

A step fails live and has no recording: show its local evidence (B11 for payment and refund, the D6 test for the race, B12 for load) and say: *“verified locally, not yet on dev.”* Never present a local result as a dev one.

After the demo (17:01:00 – 17:04:00)

k6 pre-run for slide 36 (rubric 5.3)

- No pre-run (decided 24 Oct): slides 26 and 36 keep the 16 Oct ECS run (run 20261016-v2-monolith-repeat, #670); it is complete and evidenced, and a new run would need an ECS session for little gain.
- Slide 36’s table shows the 16 Oct run (#670). On the day the report file is opened from Eric’s machine if asked. Never point k6 at the SWE5006 dev host.

Slide 36 Performance against the target: the scripted k6 run

17:00:00–17:02:00 · 120 s

Speaker Yorik · covers guideline item **5b**

Say

This saved k6 test ran from ten to two hundred requests per second. Every step passed. The worst p95 was fifty-three milliseconds, with no errors or dropped requests. These results are from our separate ECS setup, not the current dev host.

≈ 41 words · ≈ 19 s at 130 wpm · slot 120 s · if short of time: If over time, drop "with no errors or dropped requests"; the cues and Q&A keep the limits.

Cues

- Pre-run (rubric 5.3: a live run over ≈ 2 min is pre-run): k6 1.6.1 perf/k6/browse.js against the SWE5001 CloudFront d3lvq02o2e7tcn.cloudfront.net, ten 60 s plateaus 10 → 200 iterations/s (≈ 10 min plus setup). Shown: run 20261016-v2-monolith-repeat, Fri 16 Oct 04:59:44–05:09:45 UTC, one monolith task, image 6c7a99ab (dev on 6 Oct), test sha 767d980b; perf/k6/browse.js, lib.mjs and calibrate.mjs unchanged since (git log 767d980b..origin/dev, 23 Oct). Summary docs/swe5001/evidence/20261016-http-baseline-dashboard/v2-repeat-summary.json and v2-repeat-calibration.json (#670). If an optional pre-run passes before the 25 Oct rehearsal, swap in its numbers.
- Results: p95 per plateau 53, 39, 27, 28, 23, 23, 42, 23, 23, 23 ms (10 → 200 requests/s); 57,600 workload requests plus one setup request, http_req_failed 0 of 57,601, dropped_iterations 0; VUs used at 200/s: 100 of 400; pass on every gate (p95 < 500 ms, errors < 1 %, zero dropped). Saturation not observed, so 200 requests/s is a tested lower bound, not a limit. Show the k6 summary or the dashboard tab if asked.
- Environment prerequisite: the SWE5001 environment is applied per session and torn down nightly (research-aws.md; infra/swe5001 env-up.sh). Pan brings it up only for an optional pre-run; it does not need to be up during the talk, because slide 36 shows the saved report. Never point k6 at the SWE5006 dev host (perf/k6/README.md: "Do not load-test shared SWE5006 dev").
- Mapping (slide 26): Little's law N = X(R + Z), N = 100, Z = 0.5 s (our assumption), R ≈ 0.03 s → X ≈ 189 requests/s. History, labelled: the first 16 Oct V2 run (same day, every plateau p95 ≤ 57 ms, 0 errors in 57,600 requests; README.md in the same folder); local Node workload 30 Sep, 100 closed-loop users, p95 browse 24 ms, JOIN intent 57 ms, My Orders 18 ms, 30 expiries with worst lag 58.18 s (docs/perf/sprint3-baseline.md).
- Limits to say if asked: anonymous browse only (authenticated load needs 100 real Google sessions); one monolith task, so this is not a scaling result; the ECS environment is not the SWE5006 dev host.

Slide 37 Scalability: a stated trade-off, not a claim

17:02:00–17:02:30 · 30 s

Speaker Yorik · covers guideline item **5b**

Say

For scalability, our dev system runs on one EC2 host with its database on the same host. It has no auto-scaling. We chose this setup to keep one complete system available at a low cost. Our proposal has no scalability target. The same code also supports our separate ECS setup, but that is not the dev system shown here.

≈ 59 words · ≈ 27 s at 130 wpm · slot 30 s · if short of time: If over time, drop the last sentence.

Cues

- AWS read-only 21 Oct (orchestrator): dev EC2 i-0852b6c3475b19f05 t3.large; the only EC2 Auto Scaling group is blocbuy-swe5001-nat; Application Auto Scaling targets exist only for service/blocbuy-swe5001/commerce-svc (2–6) and notification-svc (1–4), created 21 Oct 09:16 SGT. PostgreSQL 18 and Redis 8 on 127.0.0.1 of the dev host (operator guide §1).
- Why: backend ADR 0008 (one codebase, two topologies); proposal §6.3 lists maintainability, correctness under concurrency, performance, security, code quality and auditability, not scalability. Rubric 5.2: claim scalability only if auto-scaling is configured and can be triggered on stage; on dev it is neither.
- Scale path, only if asked: B10 (SWE5001 production topology) and B12 (SWE5001 V3 run: commerce 1 → 3 tasks on 19 Oct, rejected as V3 acceptance because the generator dropped iterations).

Slide 38 Maintainability: the pipeline you just saw, and the seams in the code

17:02:30–17:03:00 · 30 s

Speaker Yorik · covers guideline item **5b**

Say

Pan has just shown how a change reaches dev. It passes six required checks, a code-owner review, and a smoke test. The code is also designed for change. To add a pricing mode, we add a strategy class and an enum value. To add an action after a team outcome, we add a subscriber. The existing rules stay unchanged.

≈ 59 words · ≈ 27 s at 130 wpm · slot 30 s · if short of time: If over time, drop "The existing rules stay unchanged."

Cues

- Points back to slide 33 (rubric 5.4: do not repeat the run). If the §4 PR was not merged on air, say so and show its checks instead.
- Seams: PricingStrategy + PricingStrategyResolver (slide 21; a duplicate PricingMode fails startup); TeamSucceeded / TeamExpired subscribers (slide 19); payment providers behind the CheckoutGateway and RefundGateway ports (external-interface transition strategy, slides 10–12; we do not claim GoF Adapter for them). A new pricing mode also needs a PricingMode constant, a domain edit, so the slide says "existing rules", not "domain unedited". Proposal §6.3 "Maintainability & extensibility (primary design drivers)… new pricing schemes, payment providers, and team-resolution consequences must be addable without modifying existing domain logic". 17 ArchUnit rules keep the module boundaries on every build (ArchitectureTest.java).

Slide 39 Thank you 17:03 · Q&A

Speaker Yorik · covers guideline item **5b**

Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Cues

- 17:04:00. The buffer to 17:10 answers questions parked during the talk first (Lim reads them from the backchannel and names the owner), then new ones. Hard stop 17:10 (rubric G1).
- Note-takers (rubric G9): all five members note every lecturer question and remark verbatim during the Q&A; after the talk they collate the notes into one GitHub issue, which becomes #463 report tasks by 27 Oct.

Q&A (17:04–17:10)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why is there no auto-scaling on dev?	Dev is one EC2 host running the monolith in Docker Compose, with PostgreSQL and Redis on the host (backend ADR 0008), so SWE5006 needs one complete, always-on system at low cost. Auto-scaling exists only in the SWE5001 topology on ECS (commerce 2–6 tasks on ALB requests per target, read 21 Oct), which we do not claim for SWE5006. Limitation: the dev host is a single point of failure.	Yorik → Pan
Is the refund real?	A real Stripe Sandbox refund: the Refund API in test mode, so no money moves. Locally on 20 Sep, refund re_3UHbeV… reached succeeded and a recheck created no duplicate (B11). On dev on 23 Oct, a natural expiry closed a two-seat bloc 18.9 s after its deadline and one S$22 Sandbox refund succeeded 40.6 s after it, once (PR #664, backend/docs/payment/stripe-dev-recovery-acceptance-20261023.md).	Yorik
What happens if Stripe is down?	The Checkout attempt is saved before Stripe is called, so a timeout leaves one recoverable attempt with a stable provider key, never a second payment (StripeCheckoutFlowTest). Refunds stay as durable obligations; the worker retries every 30 s with the same idempotency key, and after 23 hours it only searches Stripe and hands mismatches to a person.	Yorik
How do you avoid a double payment?	A repeated Checkout request with the same buyer, intent and Idempotency-Key reuses the payment. Webhook events are stored by event ID, so a replay changes nothing (StripeLocalIntegrationTest.eightConcurrentWebhookReplaysStillProduceOneMemberAndOneOrder). When paid buyers race for the last seat, one gets it and every other gets a refund obligation (StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist, local).	Yorik → Eric
Why did the demo not show notifications?	By decision: the dev host runs no message broker, so the outbox writer, relay and consumer are off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). Notifications run on the SWE5001 environment through SNS and SQS, and locally and in CI against ElasticMQ; slide 20 shows the relay's Template Method.	Pan
Why not race two buyers for the last seat on dev?	Because a live race is not repeatable on shared dev and would leave a refund obligation on a real account. The race is a test that runs on every pull request that changes the backend: twenty paid buyers race for the last seat of a team of two through the production intent and fulfilment ports against PostgreSQL 18 in Testcontainers; the team ends with 2 members, 2 orders, current_size 2 and 19 refund obligations (StripeLocalIntegrationTest.twentyBuyersRaceForLastSeatAndNineteenRefundObligationsPersist). It is a local test, and we say so (step D6).	Yorik → Eric
How do you know expiry happens within one minute on dev?	By design, by a local load measurement and by one dev measurement. TeamExpirySweepJob runs every minute (fixedDelay PT1M) under a database lease and expires every open team whose deadline has passed; the local run of 30 Sep measured a worst lag of 58.18 s for 30 teams under 100-user load. On dev on 23 Oct one bloc expired 18.9 s after its deadline and was refunded 40.6 s after it, read from the database (backend/docs/payment/stripe-dev-recovery-acceptance-20261023.md); that is one run, not a bound. On screen the Refunded row shows the closing day, not the time ("Closed 26 Oct"), and the receipt shows the refund status, so the demo shows the outcome: the 16:05 bloc, due ≈ 16:45, is refunded by the time we look.	Yorik

Is 401 vs 403 vs 400 deliberate?	Yes. The Sa-Token interceptor checks login on every /api/v1 path except anonymous GETs of listings and invites, so no session is 401 COMMON_401_001; a signed-in buyer on an admin endpoint fails StpUtil.checkRole(ADMIN) and gets 403 COMMON_403_001; a money write without the X-BlocBuy-Request marker is 400 COMMON_400_001, and a wrong value is 403 (StripeCheckoutController.checkCsrf). Mapping: GlobalExceptionHandler.	Yorik → Liu Jin
(Questions parked during the talk)	Lim reads each parked question from the backchannel in one line and names the owner; those come first. Answer each in 30–45 s: decision, reason, evidence or limitation.	Lim

6 Timeline before, on and after the day

Before the day

Date	Do	Owner
Wed 21 Oct	Deck re-planned from the rubric; §1 drafted.	Lim
Fri 23 Oct	**Slides due** (#464). Each owner reviews their slides, cards and Q&A rows; the Sprint 4 review and retro feed slides 5 and 7.	All
Sat 24 Oct	**Full dry run on dev** of D1–D6 in order and timed: a `US34-04` bloc started ≈ 10:00 by account B expires and refunds (D4), START and JOIN on `US34-04` with both accounts (D1, D2); payment and refund ids in the day log (no secrets). **Backup videos** of both demos recorded on the final Sprint 4 build deployed to dev (Sprint 4 ended 23 Oct), 1080p, one clip per step, stored in the cloud and on a USB drive (rubric G6). Optional **k6 pre-run** on the SWE5001 ECS environment (slide 36 already shows the 16 Oct run, #670). **Go/no-go** for the live payment steps after the dry run, posted in #464.	Yorik, Eric (system); Pan (pipeline, k6); Yorik decides the go/no-go with Lim
Sun 25 Oct	**Timed rehearsal** in Zoom at the real clock with injected questions; every diagram slide and its crops checked on a projector or TV from across the room (rubric V7); every rubric checklist box ticked; every marker resolved; rebuild deck and script.	All; Lim times

Presentation day (Mon 26 Oct)

Time (SGT)	Do	Owner
≤ 12:00	Health and recent runs (A.1); listings check; §5 preconditions list started. After 11:41, the scheduled `dast-baseline` run checked for slide 30 (24 Oct dispatch 40107924516 green).	Yorik, Pan
14:30	Tech check in a test Zoom: mic, camera, sharing, Do Not Disturb.	All
15:00	Merge freeze on `dev` until 17:15.	Lim
15:30	Demo branch rebased onto the final `origin/dev` and its PR opened (checks done by ≈ 15:40; no approval yet); pin the backend PR and deploy runs in tabs 3 and 4. Check the §4 and §5 preconditions lists.	Pan (§4); Eric, Yorik (§5)
≈ 16:05 (≤ 16:07)	Start the expiry bloc on `US34-04` as account B and pay; bookmark the receipt. `EXPIRY OK` at 16:10, `REFUND OK` / `REFUND LATE` at 16:50.	Yorik
16:30	Start; follow the run sheet. All five note every lecturer question and remark verbatim (rubric G9).	Lim; all take notes
17:04 / 17:10	Slide 39; parked questions; end.	Lim

After the presentation

- After the talk: the five members collate their notes into one GitHub issue; every lecturer remark in it becomes a #463 report task by 27 Oct (rubric G9).
- Day log under `docs/agile/sprint-04/log/`: the demo PR, run links and any payment or refund ids shown (no secrets).

7 Open items before the 25 Oct rehearsal

There are 0 `[FILL]`, 0 `[VERIFY]` and 0 `[CONFIRM]` markers in this script. **FILL** is a value that does not exist yet, so never present a guess; **VERIFY** is a claim to check against the repository, the report or run logs; **CONFIRM** needs team agreement. All must be gone after the 25 Oct rehearsal.

#	Item	Owner	Blocks slide
1	Paid flow on dev: START and expiry refund shown on dev 23 Oct (PR #664, merged 24 Oct, bef39b72; backend/docs/payment/stripe-dev-recovery-acceptance-20261023.md), a second buyer's paid JOIN shown on dev 24 Oct (system demo backup recording, #657). #590, #517 and #223 closed by Yorik 25 Oct. Since the 25 Oct rehearsal no payment is made on air: D1 and D3 show the 24 Oct purchase on dev, D2 its recording, D4 the 16:05 bloc. Open: the 26 Oct morning checks in the run book	Yorik	4, 5, 7, 35
2	Backup videos: both recorded on dev on 24 Oct and shared in the team chat (swe5006-ci-cd.mp4, swe5006-system.mp4). Open: collect the higher-resolution originals (shared copies are 832 × 480, rubric V7 asks for 1080p); rubric 4.6's USB copy is replaced by a copy on each presenter's laptop	Pan (pipeline), Yorik and Eric (system)	33, 35
5	Sprint 3 retrospective: closed for Sprint 3. docs/agile/sprint-03/retro.md (24 Oct, #451), reconstructed from the review and day logs, is cited by slide 7's 3 → 4 row and B1; the Sprint 4 retrospective stays open in item 6 (#619)	Lim	—
6	Sprint 4 review and retro after the talk, with its remarks (#619, decided 24 Oct); the v0.1.0-sprint4 tag was cut on 24 Oct at fba3cfbd and is on slide 5	Eric	5, 7
7	No numbered backend ADR for: the auth change from JWT to the Sa-Token session (open decision 6); MyBatis instead of the proposal's JPA; the scheduling design; the concurrency approach (open decision 3); the team lifecycle as a transition table instead of the proposal's State; the DevSecOps changes from proposal §5.2 (OWASP Dependency-Check replaced by Trivy deps, Trivy IaC and deps not required checks, three OIDC roles split per job instead of plan/deploy/apply, two IAM users with one access key against "no static credentials"). Not writable before 26 Oct: state the gap in report Appendix B (ADR index and the decisions without an ADR, #681), and write ADRs only if the team chooses	Liu Jin, Eric, Pan	4, 10
9	Dev host security group allows SSH (22) from 0.0.0.0/0 although SSM sessions work (AWS read-only, 21 and 24 Oct: sg-04358da6b0bc71343 still allows tcp 22 from 0.0.0.0/0): decided 24 Oct: close it now (nothing uses it; deploys and shells go through SSM)	Pan	4, 31
11	Re-check slide 4 status on 25 Oct (dev health, gaps); #517, #223 and #590 closed 25 Oct	Lim	4
14	Live demo change decided 24 Oct: home page heading "Buy better together" → "Buy better, together", author Pan, approver Lim; frontend only; Pan rebases the branch onto the final dev after the 15:00 freeze and opens the PR at ≈ 15:30, so its checks finish before the talk	Pan, Lim	33
17	Recount the test numbers for slide 25 on 25 Oct	Liu Jin	25
18	Reconcile frontend ADR 0004 and the EC2 operator guide with the post-merge rebuild in ci-frontend.yml and ci-backend.yml	Pan, Lim	28, 29
20	System demo preconditions for 26 Oct: Checkout and scheduling on for dev (BLOCBUY_SCHEDULING_ENABLED, #223); the 16:05 US34-04 bloc started by the Joiner account; both Google accounts signed in on Eric's machine; the 24 Oct receipts and the Filled bloc still on dev; the 24 Oct recording cued at D1's Checkout form and at clip D2; the saved CI run of the D6 test pinned	Yorik, Eric	34, 35

A Appendix — check commands

Every command is read-only. Run them off the shared screen, from a checkout of `dev` at the repository root.

A.1 Health and recent runs

```
curl -s https://d1b3np88z657cp.cloudfront.net/api/actuator/health          # expect "status":"UP"
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-frontend.yml --branch dev --limit 3
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow ci-backend.yml --branch dev --limit 3
gh pr list --repo PPsyrius/iss-groupbuy-team8 --base dev --state open      # what could still merge before th
e freeze
```

B Appendix — backup slides (questions only)

Backups follow slide 39 in the HTML deck and the PDF. In the deck press **B** for the index (BX) and click a row, or open `#B6` directly.

ID	Slide	Answers
B1	Agile artefacts appendix	Where are the backlog, stories, sprint plans, reviews and retros?
B2	Sprint 2 and 3 burndowns	How did earlier sprints burn down?
B3	ADR index	Which decisions are recorded, and where?
B4	Database schema, all tables	Which tables does each context own?
B5	Expiry and refund sequence	How does an expired team become refunds?
B6	UC6 refund recovery: candidates and decisions	How do you never refund twice?
B7	Test inventory	How many tests of each kind, and where do they run?
B8	Workflows, required checks and timings	What runs when, how long, and how often is it green?
B9	Known gaps and limitations	What is not done yet in the pipeline and on dev?
B10	SWE5001 production topology	What would scaling out look like?
B11	Stripe Sandbox refund, local evidence (20 Sep)	Is the refund real, and where has it been proven?
B12	Scale-out and load results (SWE5001 ECS + local baseline)	How does it scale, and how fast is it?
B13	UC9 listing statechart	Which listing moves are legal in each state?
B14	UC4 design diagrams, complete	Can we see every class and message of the join?
B15	Every member's use case and design problem	Where are the other members' models?
B16	Entity-relationship diagram, all tables	What does the whole schema look like?
DX	Diagram details, 1:1 (D slide-step, e.g. D15-2)	Every crop the talk steps through, with its caption, readable in the PDF
FX	Diagrams at full size	Every diagram in the deck at 1:1, scrollable